



LIFE OS

Engineering Handbook

A personal iOS app that unifies every signal from the devices you already wear and carry (Apple Watch, iPhone, Whoop, Renpho) and reasons over all of it to say what's working and what isn't. This handbook is the single reference for how it is built, why it is built that way, and what it does when things go wrong.

REVISION D 25 AUGUST 2026 IOS 26 · SWIFT 6 ARCHITECTURE: MVVM STATUS: V1 IN BUILD
AUDIENCE: PERSONAL

Table of contents

Every line is a link as well as a page number. Chapters 4 to 10 are the features, one per tab.

1	Architecture	3	8	Plan	36
1.1	System context	3	8.1	Intent	36
1.2	Module map	3	8.2	Files and folders	36
1.3	The DailyMetrics spine	4	8.3	Model	37
1.4	Local first topology	5	8.4	ViewModel	38
1.5	Conflict resolution	6	8.5	View	39
2	MVVM and project structure	7	8.6	Edge cases and tests	39
2.1	Why MVVM here	7	9	Settings	40
2.2	What each layer owns	8	9.1	Intent	40
2.3	The screen first directory layout	9	9.2	Files and folders	40
2.4	Anatomy of a feature folder	10	9.3	Model	41
2.5	Shared components	11	9.4	ViewModel	41
2.6	Rules and review checklist	12	9.5	View	42
3	Design system	14	9.6	Edge cases and tests	42
3.1	Principle	14	10	The Coach	43
3.2	Tokens	14	10.1	Intent	44
3.3	Component inventory	15	10.2	Files and folders	45
3.4	Dark mode	15	10.3	Model	46
4	Onboarding	16	10.4	ViewModel	46
4.1	Intent	16	10.5	View	47
4.2	Files and folders	17	10.6	Edge cases and tests	47
4.3	Model	17	11	Integrations	48
4.4	ViewModel	18	11.1	Apple HealthKit	48
4.5	View	19	11.2	Whoop	49
4.6	Edge cases and tests	20	11.3	Supabase	53
5	Today	22	11.4	Foundation Models	55
5.1	Intent	22	11.5	Screen Time	56
5.2	Files and folders	23	12	Data reference	58
5.3	Model	23	12.1	DailyMetrics	58
5.4	ViewModel	24	12.2	UserGoals	59
5.5	View	25	12.3	Source records	59
5.6	Edge cases and tests	25	12.4	Goal evaluation	59
6	Body	27	12.5	Store protocols	60
6.1	Intent	27	13	Production edge cases	62
6.2	Files and folders	28	13.1	Time and calendars	62
6.3	The hub	28	13.2	HealthKit failures	62
6.4	Activity, Weight and Recovery	29	13.3	Whoop failures	63
6.5	Wellness and the journal	30	13.4	Sync failures	64
6.6	Edge cases and tests	31	13.5	UI and data display	64
7	Money	32	14	Operations	66
7.1	Intent	32	14.1	Setup runbook	66
7.2	Files and folders	32	14.2	Secrets	67
7.3	Model	33	14.3	Testing strategy	67
7.4	ViewModel	34	A	Product thesis	72
7.5	View	34	A.1	The thesis	72
7.6	Edge cases and tests	35	A.2	Product PR FAQ	73
			A.3	Roadmap	73
			B	Decision log	76

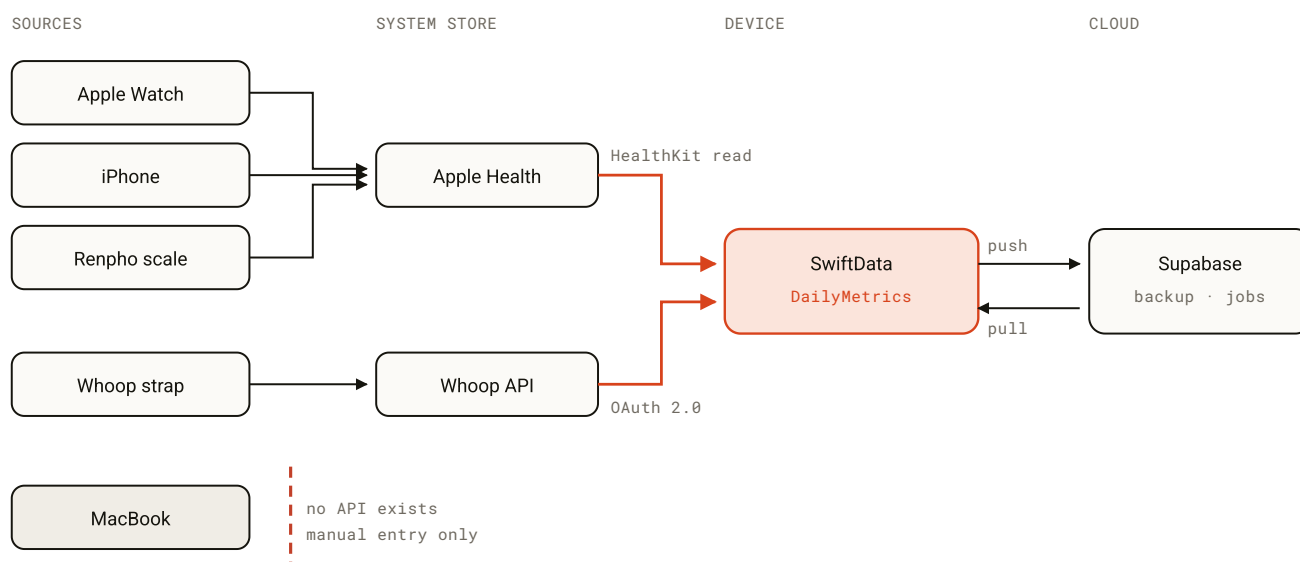
1 Architecture

Life OS is a local-first iOS app built on one bet: **the value is in the join, not the collection.** Every architectural decision below follows from it. Four devices produce data, all of it converges on a single table keyed by calendar day, and every screen reads from that table and nothing else.

This document describes a system that is designed but not finished. V1 is in active implementation. Where the code and this handbook disagree, the code is right and this handbook is a bug. The product reasoning that led here is in Appendix A; it is deliberately behind the engineering, not in front of it.

1.1 System context

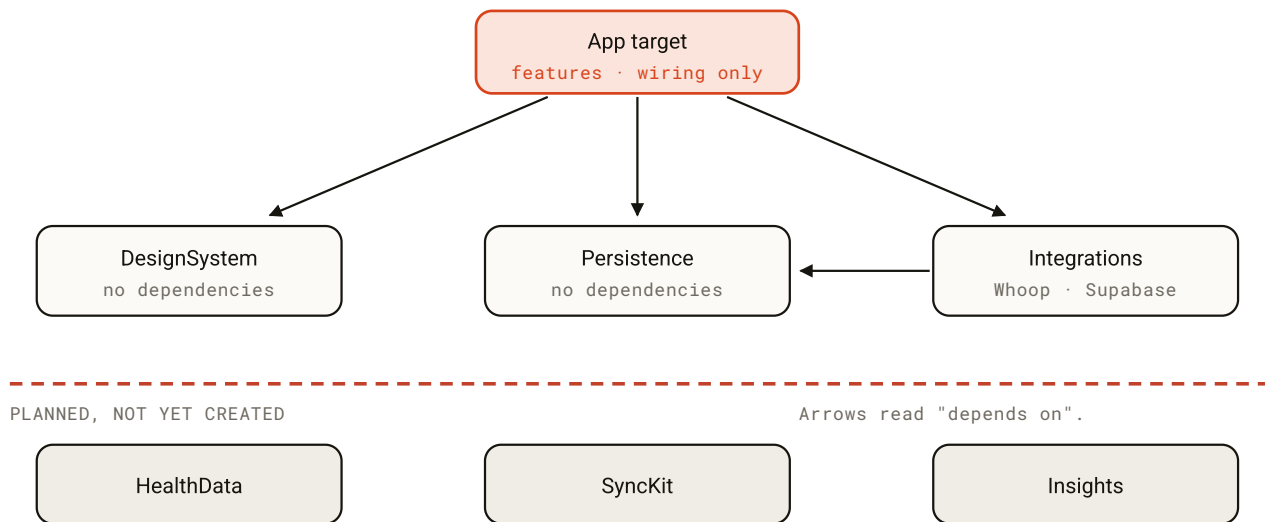
Four devices produce data, two of them reach the app through Apple Health, one through a REST API, and one (your MacBook) cannot reach it at all. Everything converges on a single local store before anything renders.



Every automatic source converges on one local table before anything renders. The MacBook has no programmatic path at any privilege level. That is a platform fact, not a missing feature.

1.2 Module map

`LifeOSKit` is a local Swift package providing compile-time boundaries. The app target contains only features and wiring, laid out per screen as described in Chapter 2. The dependency rules are enforced by the compiler, not by convention.

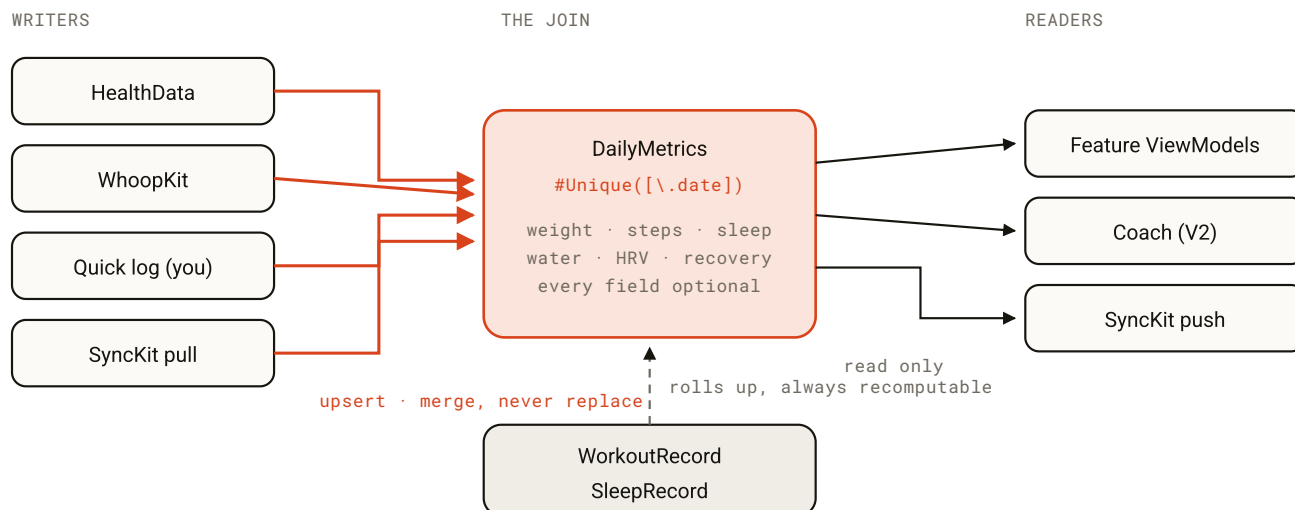


Three targets exist today. `Integrations` holds both the Whoop client and Supabase auth, and is the only one of the three that depends on another. The targets below the line are specified in this handbook and have no code: HealthKit reading, sync and the coach all still live in the plan, not the package.

RULE	ENFORCED HOW	WHY
Nothing outside <code>Integrations</code> knows Whoop exists	Package dependency graph	A vendor change is contained to one target
Nothing outside <code>Integrations</code> talks to Supabase	Package dependency graph	The app works identically with the network off
<code>Integrations</code> reads <code>Persistence</code> , never the reverse	Package dependency graph	The store cannot acquire a network dependency by accident
<code>DesignSystem</code> depends on nothing	Package dependency graph	Components stay previewable and reusable across every module
<code>DesignSystem</code> builds for macOS too	<code>Package.swift</code> platforms	<code>swift test</code> runs from the terminal without booting a simulator
No feature folder imports another feature folder	Review; see 2.6	Screens compose only through <code>Components</code> and <code>DesignSystem</code>

1.3 The DailyMetrics spine

This is the single most consequential decision in the system. **One row per calendar day.** Every source writes into it; every reader reads from it.



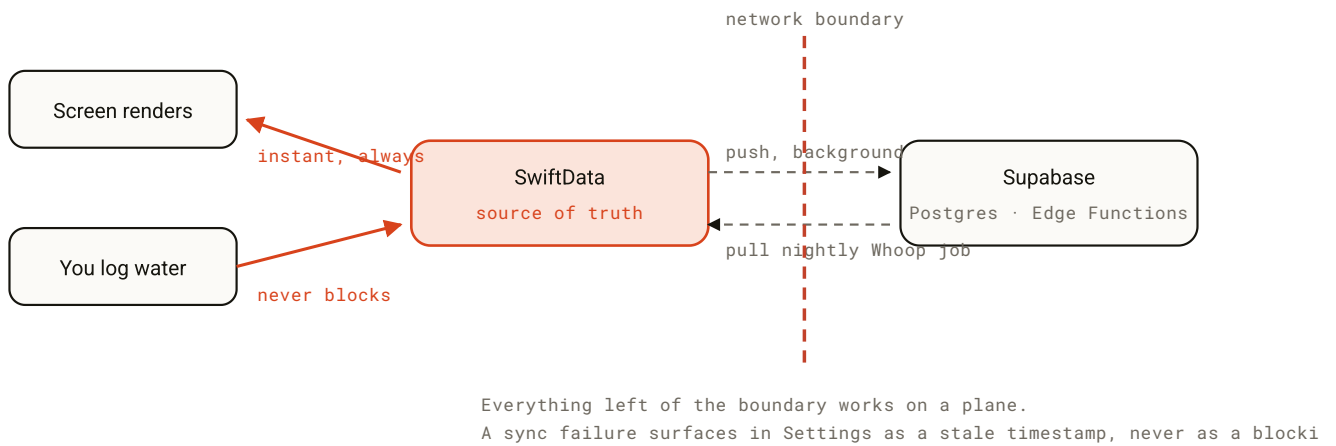
Sources merge into the row rather than replacing it, so HealthKit and Whoop writing the same day never clobber each other. **DailyMetrics** is derived state: it can always be rebuilt from the source records beneath it.

WHAT THE SINGLE-TABLE CHOICE BUYS

- **Trends become a sort**, not a multi-source asynchronous join.
- **The coach queries one table** instead of orchestrating two network APIs mid-conversation, which is what makes a 3B on-device model viable.
- **Offline works by default**, because nothing renders from a live network call.
- **Adding a source later** means writing into the same row, not building a new pipeline.
- **Every ViewModel has the same dependency**: one store protocol, so screens stay uniform and stubbable.

1.4 Local first topology

SwiftData on-device is the source of truth for the UI. Supabase is backup, sync, and a place to run jobs that a sleeping phone cannot.



Reads and writes both terminate locally. The dashed sync paths are the only network-dependent edges in the entire app, and neither is on a user-facing critical path.

1.5 Conflict resolution

Field-level last-write-wins, using `updatedAt` . Not row-level.

Row-level LWW would be a bug here. The nightly Whoop job writes recovery fields at 04:00; if you logged water at 23:00 the night before, a row-level overwrite silently discards it. Field-level merge means each writer touches only its own columns.

This is defensible only because there is exactly one human user and the writers rarely contest the same field: HealthKit owns the health columns, the Whoop job owns the Whoop columns, and you own the manual ones. If this app ever became multi-user, this strategy would need replacing outright.

KNOWN LIMITATION

LWW depends on clock agreement between the device and the Edge Function. Device clock skew of more than a few minutes can cause a stale write to win. Accepted for a single-user personal app; documented so it isn't rediscovered as a mystery bug.

2 MVVM and project structure

Every screen in Life OS is built the same way: a Model of value types, a ViewModel that owns all state and every decision, and a View that renders and forwards intent. The folder layout mirrors that exactly, one folder per screen, so a file's path tells you its layer before you open it.

2.1 Why MVVM here

SwiftUI will happily let a view query the database. `@Query` in a `View` is one line and works immediately, and it is the reason this app does not do it. Three specific things break:

- **Nothing is testable without a simulator.** Goal evaluation, streak arithmetic, and weekly deltas are the parts most likely to be wrong, and they would live inside a view body that only a running device can exercise.
- **Derivation gets re-run per render.** Status-per-day, the month grid, the streak, computed in `body`, recompute on every state change, including ones that have nothing to do with them.
- **Persistence types leak into layout code.** Once a view knows what a `@Model` class is, changing the schema means editing views.

MVVM fixes all three for the price of one extra file per screen. The ViewModel does the fetching and all derivation once, hands the View a plain value type, and can be exercised in a unit test with a stub store in milliseconds.

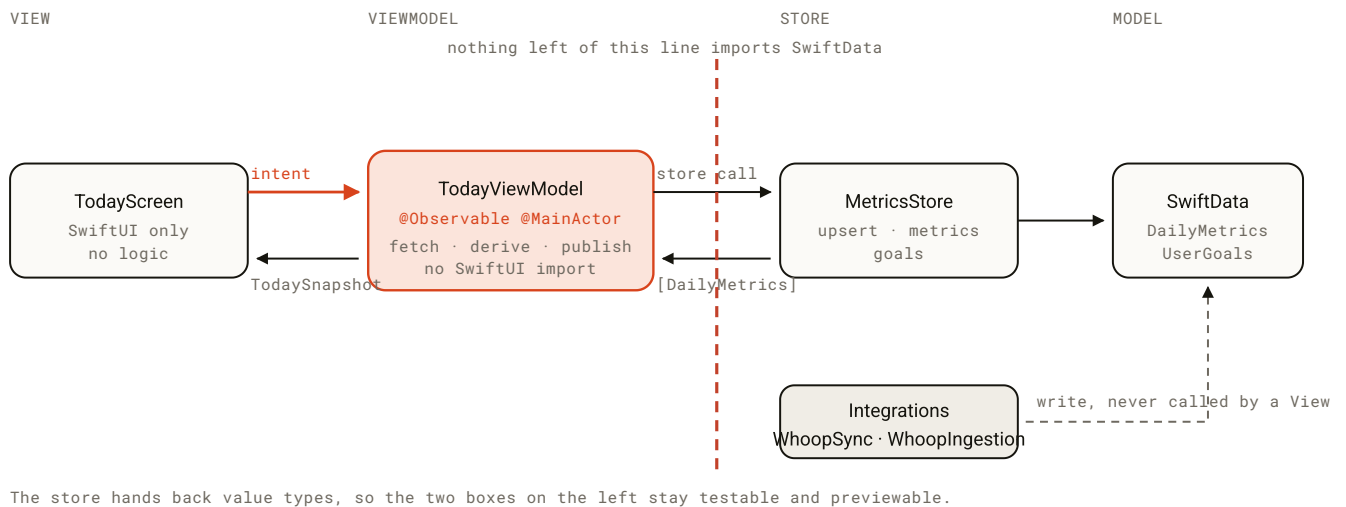
THE RULE THAT MAKES IT WORK

A ViewModel never imports SwiftUI, and a View never imports SwiftData. Both are checkable by reading the first ten lines of a file, and between them they force everything else into place.

WHERE ARE THE VIEW CONTROLLERS?

There are none, and that is not an omission. `UIViewController` is UIKit's unit of screen ownership: it exists to manage a view hierarchy's lifecycle by hand (`viewDidLoad` , `viewWillAppear` , manual reloads). SwiftUI replaces that whole job with declarative views that re-render from state, so the controller's two remaining responsibilities split cleanly: *owning state and making decisions* went to the ViewModel, and *describing what is on screen* went to the View struct. If you come looking for the controller layer, the ViewModel is where it went, minus the lifecycle boilerplate that SwiftUI made unnecessary.

2.2 What each layer owns



The View sends intent and renders a snapshot. The ViewModel is the only place that decides anything. The store is a protocol, which is what lets a test replace it with four lines of stub.

LAYER	LIVES IN	SHAPE	MAY IMPORT
Domain model	LifeOSKit/Sources/Persistence	<code>@Model</code> classes, store, pure logic	Foundation, SwiftData
Feature model	Features/<Screen>/Model	<code>struct</code> , <code>Sendable</code> , <code>Equatable</code> : the screen's rendered state	Foundation, Persistence value types
ViewModel	Features/<Screen>/ViewModel	<code>@Observable</code> , <code>@MainActor final class</code>	Foundation, Observation, Persistence; never SwiftUI
View	Features/<Screen>/View	<code>struct: View</code> , plus its private subviews	SwiftUI, DesignSystem, its own ViewModel; never SwiftData
Shared component	Components/{Model, ViewModel, View}	Same three shapes, reused by two or more screens	As above
Design primitive	LifeOSKit/Sources/DesignSystem	Stateless <code>View</code> structs and tokens	SwiftUI only; knows nothing about the app

The distinction between the last two rows is the one people get wrong. `DesignSystem` holds things that would make sense in any app: `HeroNumeral`, `DotGrid`, `GlassCard`. `Components` holds things that are reusable but specifically about *this* app's data: the quick-log sheet, the connection row, the metric tile that knows a missing value renders as an em dash.

2.3 The screen first directory layout

Folders are named after screens, not after layers. `Features/Body/` holds everything the Body screen is, and inside it the three MVVM folders classify each file. Finding code starts with "which screen," which is how anyone actually thinks about a bug report.

```
LifeOS/                                app target, file-system synchronised
├── App/
│   ├── LifeOSApp.swift                @main, container injection
│   ├── AppShell.swift                 onboarding or app; owns the URL callbacks
│   └── RootView.swift                 four tabs, FAB, sheets, view model ownership
├── Features/                          one folder per screen
│   ├── Onboarding/
│   │   ├── Model/OnboardingModels.swift
│   │   ├── ViewModel/OnboardingViewModel.swift
│   │   └── View/IntroScreen.swift, SignupScreens.swift, ConnectionsScreen.swift
│   ├── Today/
│   │   ├── Model/TodaySnapshot.swift
│   │   ├── ViewModel/TodayViewModel.swift
│   │   └── View/TodayScreen.swift
│   ├── Body/                          the Body tab, hosting four sections
│   │   ├── Model/BodySection.swift, BodySnapshot.swift, WellnessSnapshot.swift
│   │   ├── ViewModel/BodyViewModel.swift, WellnessViewModel.swift
│   │   └── View/BodyHubScreen.swift, BodyScreen.swift, WellnessSection.swift
│   ├── Activity/                      a Body section, own three layers
│   ├── Recovery/                     a Body section, own three layers
│   ├── Money/
│   │   ├── Model/MoneySnapshot.swift
│   │   ├── ViewModel/MoneyViewModel.swift
│   │   └── View/MoneyScreen.swift
│   ├── Plan/
│   │   ├── Model/PlanSnapshot.swift
│   │   ├── ViewModel/PlanViewModel.swift
│   │   └── View/PlanScreen.swift, ContentCalendar.swift
│   └── Settings/
│       ├── Model/AppConfig.swift, GoalsDraft.swift
│       ├── ViewModel/SettingsViewModel.swift, WhoopConnectionViewModel.swift
│       └── View/SettingsScreen.swift
├── Components/                        shared across two or more screens
│   ├── ViewModel/
│   │   └── QuickLogViewModel.swift
│   └── View/
│       ├── QuickLogSheet.swift
│       ├── AddEntrySheet.swift        AddPlanEntrySheet, AddMoneySheet
│       └── JournalEntrySheet.swift
└── Resources/
    └── Assets.xcassets

LifeOSKit/                             local Swift package, no UI state, no screens
```

└ Sources/	
├ DesignSystem/	tokens + stateless primitives
├ Persistence/	@Model types, stores, pure logic
└ Integrations/	Whoop and Supabase, depends on Persistence

`Coach/` is absent because it is V2 and nothing has been built. Chapter 10 specifies it; the folder appears when the code does.

WHY THIS WORKS WITHOUT TOUCHING THE PROJECT FILE

The app target uses a `PBXFileSystemSynchronizedRootGroup`. Any folder created under `LIfеOS/` is picked up by Xcode automatically, because the directory structure *is* the project structure, and no one has to remember to add files to a target. Moving a screen between folders is a `git mv`, with no project file to merge.

THE COROLLARY: DON'T PUT DOCUMENTATION IN THESE FOLDERS

Synchronised membership means *every* file is a build input. A `README.md` in each folder is copied into the app bundle as a resource, and twenty-three of them collide on the same destination path: `Multiple commands produce .../LIfеOS.app/README.md`. Empty folders are held open by a hidden `.gitkeep`, which Xcode ignores; the layer contracts are documented here instead.

2.4 Anatomy of a feature folder

Every feature folder answers the same four questions in the same order. Chapters 4 to 9 are each one instance of this template.

FOLDER	CONTAINS	TEST
<code>Model/</code>	One snapshot struct per screen, plus any small enums it needs. Value types only. No methods that fetch, no dates computed from <code>.now</code> .	Compiles without SwiftUI or SwiftData
<code>ViewModel/</code>	Exactly one class. Holds <code>private(set) var snapshot</code> , a <code>loadState</code> , the store dependency, and the <code>func</code> s the view calls.	Instantiable in a unit test with a stub store
<code>View/</code>	The screen struct and its private subviews. Reads only from its view model; owns <code>@State</code> for presentation concerns like sheet visibility and focus.	Every subview has a <code>#Preview</code> in both schemes

The ViewModel shape is identical everywhere, which is the point:

```

@MainActor @Observable
final class BodyViewModel {
    private(set) var snapshot = BodySnapshot()
    var selectedDate: Date = .now

    private var context: ModelContext?
    private let calendar: Calendar

    init(calendar: Calendar = .current) { self.calendar = calendar }

    func attach(_ context: ModelContext) { self.context = context }
    func select(_ date: Date) { selectedDate = date; load() }
    func load() { ... }
}

```

Nine view models follow this shape exactly. The context arrives through `attach` rather than the initialiser because `RootView` constructs them all as `@State` before the environment is available, and a snapshot with its empty default is a valid thing to render in the meantime.

INJECTION IS BY CONTEXT TODAY, NOT BY PROTOCOL

Every view model builds a concrete store from a `ModelContext`, so testing one means standing up an in-memory `ModelContainer` rather than passing a stub. The protocol surface that fixes this is specified in 12.5 and has not been built. It is why the package has 219 tests and the feature layer has none.

There is no shared `LoadState`. A load either replaces the snapshot or trips an `assertionFailure` and leaves the previous one in place: local SwiftData reads do not fail in ways a user can act on, and a spinner vocabulary for an operation that takes a millisecond would be ceremony. Network-backed state lives on the one view model that has any, in 9.4.

2.5 Shared components

`Components/` is split into the same three folders, for the same reason: a shared piece of UI that owns state has a view model, and it belongs somewhere obvious.

COMPONENT	MODEL	VIEWMODEL	USED BY
<code>QuickLogSheet</code>	Fields on the view model	<code>QuickLogViewModel</code> : writes water and weight	Root FAB, from any tab
<code>AddPlanEntrySheet</code>	Prompt, target and due-date flags	None, stateless	Plan, all four sections
<code>AddMoneySheet</code>	Merchant, amount, direction, category	None, stateless	Money
<code>JournalEntrySheet</code>	Text only	None, stateless	Body, wellness section

Everything stateless and reusable sits a level lower, in `DesignSystem`: `StatTile`, `MetricTile`, `SolidCard`, `GlassCard`, `HeroNumeral`, `SegmentedPill`, `WeekStrip`, `DotGrid`, `MonthCalendarView`. `Components/` holds only the pieces that need the app's own model types or write to the store.

A component earns its place in `Components/` the second time it is needed, not the first. The first time, it is a private subview inside the feature that needs it. Promoting it is a file move between two folders that already exist.

WHERE THE EM-DASH RULE LIVES

"A missing metric renders as an em dash, never as `0`" is the single most repeated rule in this app. It is enforced by type: `StatTile`, `MetricTile` and `HeroNumeral` all take `String?` and render `value ?? "-"` themselves. A screen cannot pass a zero it does not have, because it has nothing to pass. A rule stated in a handbook and reimplemented at nine call sites is a rule that will be broken at one of them.

2.6 Rules and review checklist

RULE	ENFORCED HOW	WHY
A View never imports <code>SwiftData</code>	Review: visible in the imports	All reads go through a <code>ViewModel</code> ; the schema can change without touching layout
A <code>ViewModel</code> never imports <code>SwiftUI</code>	Review: visible in the imports	Keeps it unit-testable and prevents <code>View</code> -shaped thinking leaking in
A View never holds a <code>@Model</code> instance	Review: screens take snapshots	Rendering cannot fault an object or trigger a fetch mid-layout
A <code>ViewModel</code> publishes value types only	Review: the snapshot struct is the surface	Nothing above the view model can mutate the database by accident
<code>ViewModels</code> should depend on a protocol, not <code>MetricsStore</code> <code>not yet</code>	Would be the compiler; see 12.5	A test could inject a stub; today it needs a container
One <code>ViewModel</code> per screen, constructed by the View that owns it	Review	<code>@State private var vm = ...</code> ties lifetime to the screen
No feature folder imports another feature folder	Review	Sharing goes through <code>Components</code> , which forces the shared thing to be named
Derivation happens in the <code>ViewModel</code> , never in <code>body</code>	Review	<code>body</code> runs far more often than data changes
Snapshot types are <code>Equatable</code>	Compiler	Cheap re-render skipping, and it makes <code>ViewModel</code> tests one-liners

STATUS AS OF REVISION D: THE CHAPTER DESCRIBES THE CODE, NOT A PLAN

All nine ViewModels exist and follow the shape above (Today , Body , Wellness , Activity , Recovery , Money , Plan , Settings , WhoopConnection), every screen renders a snapshot, and `grep @Query LIfе0S/Features` returns nothing. The one outstanding item is the store *protocol* surface from 12.5: ViewModels still build concrete stores from an attached `ModelContext` , which is why feature-layer tests still require an in-memory container.

3 Design system

Ten modules have to read as one app. That happens because they share a small set of primitives and differ by exactly one variable (the module hue) rather than because each screen was designed carefully on its own.

3.1 Principle

The hub is neutral; the domains are coloured. Today is a warm off-white canvas. Every domain screen is a saturated-at-top, near-white-at-bottom gradient in its own hue. This resolves two competing visual languages into one hierarchy instead of a collision, and it means adding a module later is a token, not a design exercise.

3.2 Tokens

TOKEN	VALUE	MEANING
<code>ModuleHue.body</code>	Teal	Weight and composition
<code>ModuleHue.activity</code>	Amber	Movement
<code>ModuleHue.recovery</code>	Blue	Whoop, HRV, sleep quality
<code>ModuleHue.nutrition</code>	Purple	V3
<code>ModuleHue.money</code>	Green	V6
<code>ModuleHue.habits</code>	Orange	V4
<code>LifeOSTokens.accent</code>	Orange, fixed	Exactly two meanings: <i>today</i> , and <i>act</i>
<code>LifeOSTokens.canvas</code>	Warm off-white	The neutral hub

The accent never varies by module. It marks the current day, the FAB, the selected tab, and alert states. Everything else on screen is the module hue or a neutral. Two colours competing for "this is important" is the fastest way to make neither mean anything.

Colours are `AdaptiveColor` pairs resolved explicitly by the consuming view rather than UIKit dynamic colours, because the package also builds for macOS so that `swift test` runs from the terminal without a simulator. Explicit resolution is the price of fast tests.

3.3 Component inventory

PRIMITIVE	WHAT IT IS	NOTES
<code>GradientCanvas(hue:)</code>	The one gradient primitive: saturated top → near-white bottom	Adding a module is a hue token, not a new screen design
<code>GlassCard</code> / <code>SolidCard</code>	Two card treatments by zone: frosted over the saturated region, solid with a soft shadow once the background has gone pale	What makes cards legible at both ends of a gradient
<code>DotGrid</code>	The shared vocabulary of the whole app: filled = target met, hollow = missed, orange = today, grey = future, faint outline = no data	Month view on Today, streaks on Habits (V4), "days on target" inside any domain screen
<code>MonthGridLayout</code>	Pure calendar maths producing the grid's cells	No SwiftUI; fully unit-tested against a fixed UTC calendar
<code>HeroNumeral</code>	~96pt semibold numeral with tight tracking and a quiet label beneath	Never renders a numeral without its unit
<code>StatTile</code>	Small metric tile: label, value, optional progress	Wrapped by <code>Components/View/MetricTile</code> , which adds the app's missing-value rule
<code>WeekStrip</code>	Pinned seven-day header, today filled orange	Selection is a binding, so the owning ViewModel decides what selection means
<code>MonthCalendarView</code>	<code>WeekdayHeader</code> , <code>DateHeadline</code> , and the assembled month	Takes <code>[DotCell]</code> and a date, with no data access, so a snapshot literal previews it

The tab bar is circular icons plus a floating dark FAB for logging. **In V1 the FAB logs water and weight only**, the two metrics with no automatic source. Both write to local `DailyMetrics` through `QuickLogViewModel`; the app requests no HealthKit write permissions in V1. The FAB's menu grows in later slices (meals in V3, habits in V4).

3.4 Dark mode

Every hue has a dark variant: deep saturated top, near-black bottom, card materials inverted. These are defined as tokens in V1, before any screen exists, because retrofitting a gradient system to dark mode later is significantly harder than doing it once at the start.

NO COLOUR IS DEFINED ONLY INSIDE A SCHEME BRANCH

Every token has a light value and a dark value, always both. A colour that exists only in the dark branch renders as whatever the parent inherited in light mode, usually black on black, and usually discovered by a user rather than a preview.

4 Onboarding

Four pages of pitch, then an identity, then a name, then the connections. Signup is mandatory and it is the only entry point, so this flow is the one screen every user sees before any other.

4.1 Intent

WORKING BACKWARDS · V1 FEATURE

Life OS opens on a four-page intro, one page per life domain in tab order. Signing in takes a phone number or an email address and a code, a first and last name, and then offers Whoop and Apple Health. Every connection is skippable. Nothing on the way in asks for a password.

Why is there an account at all in a personal app?

Because the data outlives the phone. The account is what lets Supabase hold a backup and run nightly jobs, and it is what Row Level Security keys every row to. Without it, a lost device is a lost year.

Why no password?

A password is a credential to lose, store and reset. A one-time code delivered to a channel the user already controls proves the same thing with no secret at rest and no reset flow to build.

Phone or email?

Phone, when the project can send SMS. The app asks Supabase which channels are actually enabled before it renders the choice, and falls back to email when SMS is not configured, rather than offering a button that silently fails.

Why can every connection be skipped?

Because a screen that blocks on a Whoop login is a screen people abandon. Everything Life OS shows degrades to a blank rather than a wrong number, so an unconnected app is honest, not broken. Connections are offered again in Settings.

FLOW ANATOMY

- **IntroScreen** : four pages, one per domain, each carrying its own hue.
- **IdentityScreen** : channel switcher, dial-code picker, destination field.
- **CodeScreen** (phone) or **LinkSentScreen** (email).
- **ProfileScreen** : first name, last name, country.
- **ConnectionsScreen** : Whoop now, Health and Plaid marked as next release.

4.2 Files and folders

```
Features/Onboarding/
├─ Model/
│   └─ OnboardingModels.swift      step enum, draft, intro pages, dial codes
├─ ViewModel/
│   └─ OnboardingViewModel.swift   send, verify, profile, callback handling
└─ View/
    ├─ IntroScreen.swift           the four-page pager
    ├─ SignupScreens.swift         scaffold, identity, code, profile, link sent
    └─ ConnectionsScreen.swift     Whoop and Health, plus OnboardingFlow
```

`OnboardingFlow` lives beside `ConnectionsScreen` rather than in its own file because it is a twenty-line switch over `OnboardingStep` and nothing else. `AppShell` chooses between it and `RootView`, so the tab hierarchy is never constructed for a signed-out user.

4.3 Model

```
enum OnboardingStep: Equatable {
    case intro
    case identity           // phone or email
    case code               // OTP (phone)
    case linkSent           // magic link (email)
    case profile            // name, country
    case connections        // Whoop and Health, all optional
}
```

ONE ENUM, NOT FIVE BOOLEANS

`OnboardingStep` is a single value because the alternative, `hasSentCode && !hasVerified && isEmail`, can represent states that do not exist. A user cannot be on the code screen and the profile screen at once, and the type should say so.

```

struct SignupDraft: Equatable {
    var channel: SupabaseAuthChannel = .phone
    var phone = ""
    var dialCode = "+1"
    var email = ""
    var code = ""
    var firstName = ""
    var lastName = ""
    var country = Locale.current.region?.identifier ?? "US"

    var e164: String { dialCode + phone.filter(\.isNumber) }
    var destination: String {
        channel == .phone ? e164 : email.trimmingCharacters(in: .whitespaces)
    }

    var canSendCode: Bool
    var canVerify: Bool
    var canFinishProfile: Bool
}

```

The draft holds what the user typed, before any of it is trusted. The three `can...` properties are the only validation in the feature, and they live on the model rather than in a view so a disabled button and a rejected submit can never disagree.

`DialCountries.all` is built from `Locale.Region.isoRegions` rather than a hand-typed list, so it is complete and localised. A region with no known dial code is dropped rather than guessed at: offering the wrong prefix sends an SMS into the void, and the user has no way to tell that is what happened.

4.4 ViewModel

```

@MainActor @Observable
final class OnboardingViewModel {
    private(set) var step: OnboardingStep = .intro
    var draft = SignupDraft()

    private(set) var isBusy = false
    private(set) var errorMessage: String?
    private(set) var resendIn = 0
    private(set) var availableChannels: Set<SupabaseAuthChannel> = []
    private(set) var isSignedIn: Bool

    func loadChannels() async
    func restoreSession() async -> Bool
    func sendCode() async
    func verifyCode() async
    func saveProfile() async
    func handleAuthCallback(_ url: URL)
    func signOut()
}

```

The rules that live here, and nowhere else:

- **Channel availability is asked, not assumed.** `loadChannels()` reads `auth/v1/settings` and switches the default to email when SMS is off. The identity screen shows a note explaining why rather than leaving a dead button on screen.
- **Email uses a magic link, deliberately and temporarily.** The free tier will not let the email template carry a six-digit token, so `emailUsesMagicLink` is `true` and the flow diverts to `.linkSent`. Flip that one constant once custom SMTP is configured and the OTP screen returns with no other change.
- **A failed profile write does not strand the user.** The account already exists by then, so `saveProfile()` records the error and advances to `.connections` anyway. Blocking the last step of signup on a metadata write would trade a cosmetic failure for an unusable app.
- **Resend is rate-limited in the UI.** A thirty-second countdown starts on every send, because the server's own limit answers with a 429 the user cannot interpret.
- **`isSignedIn` is stored, not computed.** It was a computed `store.load() != nil`, which `@Observable` cannot see: nothing told SwiftUI the value had changed, so the shell could keep showing the app after a sign-out until some unrelated edit happened to redraw it. It is now written wherever the store is.
- **Restoring is one call, and it is shared.** `restoreSession()` wraps `SessionRefresher` (11.3) and holds the in-flight `Task`, so a launch and a foreground arriving together await one refresh rather than racing. That is not defensive coding: Supabase rotates the refresh token on use, so the second call of a pair would present a token the first had just retired and be refused.

THE TOKENS ARRIVE IN THE FRAGMENT, NOT THE QUERY

Supabase returns a magic-link session in the URL *fragment*. `queryItems` finds nothing there, so the callback looks empty until the fragment is parsed by hand. `AuthSession(callback:)` reads both, and `AuthSession.errorDescription(in:)` pulls `error_description` out of the same place so an expired link says so.

4.5 View

```
struct SignupScaffold<Content: View, Action: View>: View {
    let title: String
    let subtitle: String
    let onBack: () -> Void
    @ViewBuilder var content: Content
    @ViewBuilder var action: Action
}
```

Every signup step is the same shape: a back control, a title, a subtitle, one field group and one primary button. `SignupScaffold` owns that shape so the four screens differ only in their content, which is what keeps them from drifting apart visually as they are edited.

`IntroScreen` renders `IntroPage.all` in a `TabView` pager. The pages are data rather than four hand-written views for the same reason: a loop cannot let page three fall out of step with page one.

`ConnectionsScreen` is the only onboarding screen that reads live state. It takes the shared `WhoopConnectionViewModel`, so a connection made here is already made when Settings opens later, and it renders Health and Plaid as explicitly next-release rather than as buttons that do nothing.

4.6 Edge cases and tests

CONDITION	BEHAVIOUR
No <code>SUPABASE_ANON_KEY</code> in the build	<code>isConfigured</code> is false; the screen says sign-in is not configured instead of failing on tap
SMS not enabled on the project	Channel defaults to email; a note explains why phone is unavailable. Phone is enabled on the project today, so the live app opens on the phone tab
Magic link opened on a different device	No session arrives on this device; the user stays on <code>.linkSent</code> and can resend
Expired or reused link	<code>error_description</code> from the fragment is shown verbatim; flow returns to <code>.identity</code>
Profile write fails	Error recorded, flow advances to connections; the account is already valid
Returning user, session in Keychain	<code>AppShell</code> skips signup entirely on launch, after <code>restoreSession()</code> renews the session; see 11.3
Returning user, session expired	Refreshed before the shell decides anything. Only a refusal sends the user back to signup, with "Your session expired. Sign in again"
Returning user, no network	Stays signed in. The credential is still good; only the network is missing. Every screen already reads local
App foregrounded after an hour	Renewed on the <code>scenePhase</code> change. That path can sign a user out but never <i>in</i> : promoting there too would yank someone out of the profile step for switching apps mid-signup
Rate limited (429)	Supabase's own message is surfaced; the resend countdown is already running

THE GUEST BYPASS IS TEMPORARY

"Skip for now" sets `isGuest` on `AppShell`, which is deliberately *not* persisted: a relaunch returns to signup. It exists so the app can be exercised without a working Supabase project, and it must not be allowed to quietly become the default state of the app.

TESTS

Session restoration is covered: `SessionRefreshTests` drives every branch of `SessionRefresher` against a stub `URLProtocol`, which is what pins down the rule that only a refusal signs a user out. The rest of auth is still the largest untested surface in the app, and the three below are the ones worth writing next, because each covers a parsing rule that fails silently.

- `SignupDraft` validation: a six-digit code with spaces verifies, a five-digit one does not; an email without a dot does not send.

- `AuthSession(callback:)` against a real Supabase fragment: tokens parsed, expiry derived from `expires_in`, and a fragment with no `access_token` returns `nil` rather than an empty session.
- `AuthSession.errorDescription(in:)` decodes `+` as a space, so "Email link is invalid or has expired" reads as a sentence.

THE SIGNUP FLOW IS EXERCISABLE WITHOUT SENDING AN SMS

The project keeps a test-number map, so `+1 123-456-7890` with code `123123` returns a real session and never reaches Twilio. That is what makes the phone path runnable in the simulator, where the magic-link round trip cannot be.

5 Today

The hub. Neutral canvas, a month of dots, a streak, and four tiles. It answers "am I on track" before you have processed a single number.

5.1 Intent

WORKING BACKWARDS · V1 FEATURE

Today opens on a grid of dots, one per day this month, with the month and weekday header above it. Filled dots are days you hit your targets, hollow dots are days you didn't, and today is orange. Beneath it a streak count, then four tiles: steps, sleep, weight, recovery. You can read the whole month in about a second.

What makes a dot fill?

Meeting **3 of 4** daily goals: steps $\geq 8,000$, sleep $\geq 7h$, exercise ≥ 30 min, water $\geq 2,500$ ml. All five values (the four goals and the required count) are editable in Settings. The rule is deliberately explicit; a vague definition makes the entire grid meaningless.

What happens on a day with no data at all?

It renders as no-data, not as a miss, and it is **excluded from streak arithmetic entirely**. Leaving your watch on the charger is not a moral failure, and a system that punishes it teaches you to distrust the streak.

And a day with *some* data?

Judged on what it has. If steps are logged at 500 and nothing else was recorded, that's a miss. The alternative, treating partial days as blank, would let forgetting to log water silently launder a bad day into a neutral one.

If I change a goal, does history change?

Yes. Status is computed on read, never stored, so past dots re-evaluate against the new target immediately. The grid answers "how am I doing against my current standard," not "how did I do against a standard I've since abandoned."

SCREEN ANATOMY

- **Header:** day numeral at 92pt, month and year, weekday, all derived from the selected date.
- **WeekStrip:** seven days, today filled orange, selection bound to the ViewModel.
- **Month card:** month label, streak count in accent, `DotGrid`.
- **Tile grid:** steps, sleep, weight, recovery; each a `MetricTile`.

5.2 Files and folders

```
Features/Today/  
├─ Model/  
│   └─ TodaySnapshot.swift           rendered state, no behaviour  
├─ ViewModel/  
│   └─ TodayViewModel.swift         fetch, evaluate, derive grid + streak  
└─ View/  
    └─ TodayScreen.swift             layout only
```

Consumes from the package: `DotGrid`, `MonthGridLayout`, `WeekStrip`, `SolidCard`, `MetricTile`, `evaluate`, `currentStreak`.

5.3 Model

```
struct TodaySnapshot: Equatable {  
    var date: Date = .now  
    var cells: [DotCell] = []           // from MonthGridLayout  
    var streak: Int = 0  
  
    var steps: Int?  
    var stepsProgress: Double?  
    var sleepMinutes: Int?  
    var sleepProgress: Double?  
    var weightKg: Double?  
    var recoveryPct: Double?  
}
```

Every metric is an optional, and each one that has a goal carries its progress alongside it, already divided. The view receives numbers it can render and nothing it has to interpret: it never sees a `DailyMetrics`, so rendering cannot fault a `SwiftData` object or trigger a fetch mid-layout.

A SEPARATE PROGRESS FIELD, NOT A COMPUTED ONE

`stepsProgress` is `nil` exactly when `steps` is, so a day with no data draws no ring rather than an empty one. Computing the ratio in the view would need the target in the view, which would put goals in `body` and the missing-versus-zero rule in two places.

5.4 ViewModel

```
@MainActor @Observable
final class TodayViewModel {
    private(set) var snapshot = TodaySnapshot()

    private var context: ModelContext?
    private let calendar: Calendar

    func attach(_ context: ModelContext)
    func load()
}
```

Inside `load()`, in order:

1. Read `GoalTargets` from the store, which materialises defaults on first access.
2. Fetch four hundred days ending now, most-recent-first. Bounded on purpose: the grid needs this month and the streak needs recent history, never the whole table.
3. `evaluate` each row once into a date-keyed dictionary, **not once per dot**. Rebuilding that map inside the per-day closure would re-evaluate every row about thirty times per load, which is what a view-body implementation ends up doing.
4. Ask `MonthGridLayout` for the cells, mapping `DayStatus` to `DotState`. This mapping lives here because `Persistence` must not depend on `DesignSystem`.
5. Compute `currentStreak` from the same ordered statuses.
6. Publish one `TodaySnapshot`. A single assignment means a single render.

A read failure trips an `assertionFailure` and leaves the previous snapshot in place rather than blanking the screen. Nothing about a failed local fetch is recoverable by the user, so the last good state is the most useful thing to keep showing.

5.5 View

```
struct TodayScreen: View {
  let snapshot: TodaySnapshot

  var body: some View {
    ScrollView {
      VStack(alignment: .leading, spacing: 22) {
        MonthCalendarView(date: snapshot.date, cells: snapshot.cells, ...)
        streakLine
        LazyVGrid(columns: [GridItem(.flexible()), GridItem(.flexible())]) {
          SolidCard { StatTile(label: "Steps", value: ..., progress: snapshot.stepsPr
          SolidCard { StatTile(label: "Sleep", value: ..., progress: snapshot.sleepPr
          SolidCard { StatTile(label: "Weight", value: ..., unit: "kg") }
          SolidCard { StatTile(label: "Recovery", value: ..., unit: "%") }
        }
      }
    }
  }
}
```

The screen takes a value and owns nothing: no view model, no store, no context. `RootView` holds `TodayViewModel` and passes `today.snapshot` in, which is what makes a preview a single literal. Weight and recovery pass no `progress`, so they draw no ring: neither has a goal, and a progress ring with no target is a decoration pretending to be information.

No `@Query`, no `modelContext`, no arithmetic in `body`. Reloads are driven from `RootView`, which reloads every tab on `ModelContext.didSave` and on returning to the foreground, rather than each screen polling for itself.

REDESIGN IN FLIGHT

docs/superpowers/specs/2026-08-12-today-default-tab-and-interactive-dots-design.md specifies Today as the launch tab with tappable day dots. It is approved and not yet built; this section describes what ships today.

5.6 Edge cases and tests

CONDITION	BEHAVIOUR
Month boundaries	Leading blanks from <code>firstWeekday</code> , padded to whole rows, tested in <code>MonthGridLayout</code>
Future days in the current month	Rendered <code>.future</code> by position, never evaluated against goals
No data for a day	Faint outline dot; excluded from the streak
Goal changed in Settings	Next <code>load()</code> re-evaluates all history; dots change
Recovery tile before Whoop is connected	Em dash, never <code>0%</code>

TESTS

- `PersistenceTests` : evaluation and streak arithmetic (pure, terminal, no simulator).
- `DesignSystemTests` : month grid maths against a fixed UTC calendar.
- `TodayViewModel` `to be written` : an empty container produces a snapshot of `nil` s and a zero streak; a seeded month produces the expected cell count; changing targets between two `load()` calls changes the cells.

6 Body

One tab for everything the body does: movement, weight, recovery and wellness, behind a segmented switcher and a week strip. Four domains share a tab so the tab bar stays free for the rest of a life. Life OS is not a fitness app.

6.1 Intent

WORKING BACKWARDS · V1 FEATURE

Body opens on Activity: the day's steps, active minutes, energy and resting heart rate. A pill switches to Weight, Recovery or Wellness, and the canvas re-tints as you go. A week strip across the top moves every section to the same day at once.

Why is this one tab instead of four?

Because the tab bar is the app's scarcest surface, and four of its five slots going to the body would say Life OS is a fitness app. It is not. Money and Plan earn their slots; activity, weight, recovery and wellness are all answers to "how is my body", so they share one.

Why does the whole tab move to one selected day?

Because the four sections are readings of the same day. Letting each keep its own date would make it possible to read Tuesday's recovery against Thursday's training and draw a conclusion from a comparison that was never real.

Both my iPhone and Watch count steps. Does that double?

No. Steps arrive through `HKStatisticsCollectionQuery`, which de-duplicates overlapping samples across sources. Summing raw samples client-side, the obvious naive approach, would roughly double every step count on days you carried both devices.

Where does weight come from?

Your Renpho scale, via Apple Health. Renpho already syncs there, so Life OS never talks to Renpho directly, which is one less integration, one less thing to break, and no third-party credentials to store.

SCREEN ANATOMY

- `GradientCanvas(hue: section.hue)`, re-tinting as the section changes.
- `WeekStrip`: the selected day, shared by all four sections.
- `SegmentedPill`: Activity · Weight · Recovery · Wellness.
- One section view below, swapped on the selection.

6.2 Files and folders

```
Features/Body/
├── Model/
│   ├── BodySection.swift           the four sections, each with its hue
│   ├── BodySnapshot.swift         weight, weekly delta, 14 points
│   └── WellnessSnapshot.swift     week rollup, verdicts, journal
├── ViewModel/
│   ├── BodyViewModel.swift        weight and its delta
│   └── WellnessViewModel.swift    week averages, journal writes
└── View/
    ├── BodyHubScreen.swift        the hub: strip, pill, section switch
    ├── BodyScreen.swift           the Weight section
    └── WellnessSection.swift      verdict tiles and the journal card

Features/Activity/                 the Activity section, same three layers
Features/Recovery/                 the Recovery section, same three layers
```

FOUR SECTIONS, FOUR VIEW MODELS, ONE SCREEN

Activity and Recovery keep their own folders and view models even though they no longer own tabs. A section is a view concern; what each one derives is not. Merging them into one `BodyViewModel` would produce a class that loads a day, a fortnight and a week for four unrelated readings, and would have to reload all of it whenever any part changed.

6.3 The hub

```
struct BodyHubScreen: View {
    let activity: ActivitySnapshot
    let weight: BodySnapshot
    let recovery: RecoverySnapshot
    let wellness: WellnessSnapshot
    var onAddJournal: () -> Void = {}

    @Binding var section: BodySection
    @Binding var selectedDate: Date
}
```

The hub takes four value snapshots and two bindings, and owns no view model at all. That is what makes its preview a single literal and lets every section be looked at in any state without a store. `RootView` holds the selection and fans a date change out to all four view models:

```
private func selectBodyDate(_ date: Date) {
    activity.select(date)
    weight.select(date)
    recovery.select(date)
    wellness.select(date)
}
```

BodySection carries its own hue, so re-tinting the canvas is a property read rather than a switch inside the view:

```
enum BodySection: String, CaseIterable, Identifiable {
    case activity, weight, recovery, wellness

    var hue: ModuleHue {
        switch self {
            case .activity: .activity
            case .weight:    .body
            case .recovery:  .recovery
            case .wellness:  .habits
        }
    }
}
```

6.4 Activity, Weight and Recovery

```
struct ActivitySnapshot: Equatable {
    var steps: Int?
    var exerciseMinutes: Int?
    var activeEnergyKcal: Double?
    var restingHR: Double?
}

struct BodySnapshot: Equatable {
    var weightKg: Double?
    var weeklyDeltaKg: Double? // nil without 7 days of history
    var recentWeights: [WeightPoint] = [] // oldest-first, nil marks a gap
}

struct RecoverySnapshot: Equatable {
    var recoveryPct: Double?
    var hrvMs: Double?
    var dayStrain: Double?
    var sleepMinutes: Int?
}
```

Every field is optional, without exception. A day with the watch on the charger produces four **nil**s and four em dashes, never four zeros. This is the rule the whole app is built on and it is enforced by the types, not by discipline at each call site.

`ActivityViewModel` and `RecoveryViewModel` read exactly one row: the selected day, no history.

`BodyViewModel` is the one that derives anything:

- Fetch sixty days ending at the selected date, most-recent-first.
- The hero is the most recent row that actually has a weight, which is not necessarily the most recent row.
- The delta is that weight minus the most recent weigh-in at least six days older. If no such weigh-in exists, **the delta is suppressed entirely** rather than computed over a shorter window. A "weekly change" measured over two days is a wrong number wearing a right label.
- The chart takes the last fourteen rows reversed, keeping `nil` weights as points rather than filtering them out. Dropping them would silently compress the axis and make a fortnight holding two weigh-ins look like two consecutive days.

SIXTY DAYS TO FIND ONE NUMBER

The window is sixty days rather than fourteen because the delta needs a weigh-in from a week ago and people skip weeks. Fetching a fortnight would suppress the delta for anyone who steps on the scale fortnightly, which is a real user, not an edge case.

6.5 Wellness and the journal

```
struct WellnessSnapshot: Equatable {
    var averageSleepMinutes: Int?
    var workoutDays: Int = 0
    var workoutTarget: Int = 7
    var averageExerciseMinutes: Int?

    var sleepVerdict: String?      // Optimal / Fair / Low
    var trainingVerdict: String?   // Consistent / Patchy

    var journal: [JournalEntry] = []
    var hasEntryToday = false
}
```

Wellness is the one section that rolls up a week rather than reporting a day, because sleep and training only mean anything as a pattern. Seven days ending at the selected date: sleep is averaged over the days that have it, and a workout day is one with twenty minutes or more of exercise.

The verdicts are computed in the view model and carried as strings. The view renders "Optimal"; it does not know that the threshold is seven hours. A judgement implemented in a view body is a judgement that gets re-implemented, slightly differently, in the next view body.

WHY THE JOURNAL LIVES IN BODY

A journal entry is a wellness signal, not a task: how the day felt, next to how the body performed. It is stored as a `PlanEntry` of kind `.journal` with its date in `dueDate`, so it rides the same store as goals and habits and will land on the daily spine without a migration when it needs to.

6.6 Edge cases and tests

CONDITION	BEHAVIOUR
Watch on the charger all day	Every Activity reading <code>nil</code> ; em dashes, never zeros
Never weighed	Hero blank, delta suppressed, chart empty
Fewer than 7 days between weigh-ins	Delta suppressed entirely rather than shortened
Gap in the middle of the fortnight	Point kept as <code>nil</code> ; axis position preserved
Whoop not connected	Recovery, HRV and strain all <code>nil</code> ; sleep may still arrive from Health
No sleep recorded all week	<code>averageSleepMinutes</code> and <code>sleepVerdict</code> both <code>nil</code> ; no verdict is invented
Day selected in the future	Empty row, blank readings; the strip allows it and the app does not pretend otherwise
Section switched mid-load	Sections read snapshots, not stores; the switch is instant and cannot tear

TESTS

The store beneath these sections is covered by `MetricsStoreTests` : day-normalised upsert, per-source field ownership, and range queries. The derivations above are not yet covered. Worth writing, in order:

- `BodyViewModel` delta suppression: six days of history yields `nil` , eight yields the signed difference, and a fortnight with one weigh-in yields `nil` .
- Chart construction: three scattered weigh-ins in fourteen rows yield fourteen points, eleven of them `nil` .
- `WellnessViewModel` thresholds at the boundary: exactly twenty exercise minutes counts as a workout day; exactly 420 sleep minutes reads "Optimal".

7 Money

What you kept this month as the hero, income and expenses and savings rate beneath it, and the recent transactions under that. Built for Plaid from the first line, fed by hand until Plaid lands.

7.1 Intent

WORKING BACKWARDS · V1 FEATURE

Money is green. The hero is this calendar month's net, with a one-word verdict beneath it: on track, tight, or overspending. Three tiles carry income, expenses and the share of income you kept. Net worth appears only once there are accounts to sum. Transactions are added by hand today and will arrive from Plaid without a schema change.

Why is the savings rate sometimes blank rather than 0%?

Because a savings rate against no income is not zero, it is undefined. Rendering 0% would tell you that you saved nothing, when what actually happened is that nothing was logged. The whole app draws that distinction and this is the sharpest case of it.

Why manual entry when Plaid is the plan?

Because the model, the store, the rollup and the screen are the work, and none of it depends on where the rows come from. Manual entry exercises all of it now and becomes a permanent feature anyway: cash exists, and Plaid will never see it.

What happens to a pending charge?

It is stored and shown, greyed, but excluded from every rollup. Plaid re-issues a pending transaction with a new identifier when it settles, so counting it twice is the default failure here, and the rollup refusing to see pending rows is what prevents it.

7.2 Files and folders

```
Features/Money/
├── Model/
│   └── MoneySnapshot.swift           hero, tiles, rows, verdict
├── ViewModel/
│   └── MoneyViewModel.swift         month fetch, rollup, manual add
└── View/
    └── MoneyScreen.swift            hero, tiles, net worth, transactions
```

LifeOSKit/Sources/Persistence/	
└─ MoneyEntry.swift	entry, account, summary, summarise()
└─ MoneyStore.swift	range and month queries, upsert ingest

The arithmetic lives in the package, not the feature. `summarise(entries:accounts:)` is a free function over value types with no store and no context, which is why the money rules are the best-tested code in the app.

7.3 Model

```
struct MoneySnapshot: Equatable {
    var income: Double = 0
    var expenses: Double = 0
    var net: Double = 0
    var savingsRate: Double? // nil when there is no income
    var netWorth: Double?    // nil when there are no accounts
    var recent: [MoneyRow] = []
    var monthLabel: String = ""
    var isConnected = false

    var verdict: String // On track / Tight / Overspending
}
```

SIGN CONVENTION: POSITIVE IS MONEY IN

`MoneyEntry.amount` is positive for income and negative for expenses. **Plaid is the opposite:** its `amount` is positive for outflows. The ingestion layer must negate on the way in. This is written down in three places, including the type itself, because a silent sign flip turns income into expenses and every number on this screen lies without ever looking wrong.

```
public struct MoneySummary: Sendable, Equatable {
    public var income: Double = 0
    public var expenses: Double = 0
    public var netWorth: Double?

    public var net: Double { income - expenses }

    /// Nil when there is no income. Not zero. Undefined.
    public var savingsRate: Double? {
        guard income > 0 else { return nil }
        return (income - expenses) / income
    }
}

public func summarise(entries: [MoneyEntry], accounts: [MoneyAccount] = []) -> MoneySummary
```

`MoneyAccount.netWorthContribution` is the other rule worth stating: an account of type `credit` or `loan` contributes `-abs(balance)`. Plaid reports a card balance as a positive number, so summing balances naively would make a large debt read as wealth.

7.4 ViewModel

```
@MainActor @Observable
final class MoneyViewModel {
    private(set) var snapshot = MoneySnapshot()

    func attach(_ context: ModelContext)
    func load()
    func add(merchant: String, amount: Double, isIncome: Bool, category: String?)
}
```

`load()` fetches the calendar month to date plus the accounts, calls `summarise`, and keeps the eight most recent rows for display. `isConnected` is derived rather than stored: it is true when there is any entry or any account, which is what decides between the screen and its empty state.

`add` is where the sign convention is applied. The sheet asks "income or expense" as a toggle and this method negates, so no caller below it ever has to guess at a bare number's direction.

7.5 View

```
GradientCanvas(hue: .money) {
    HeroNumeral(value: money(snapshot.net), label: "Net · \$(snapshot.monthLabel)")
    Text(snapshot.verdict)                                // capsule under the hero

    HStack {
        MetricTile(label: "Income",    value: money(snapshot.income))
        MetricTile(label: "Expenses",  value: money(snapshot.expenses))
        MetricTile(label: "Saved",     value: percent(snapshot.savingsRate), unit: "%")
    }

    if let netWorth = snapshot.netWorth { SolidCard { /* net worth row */ } }
    SolidCard { /* RECENT, with an Add button */ }
}
```

A pending row renders at 45% opacity, which is the only visual difference between it and a settled one. Income is green; expenses take the primary ink rather than red, because a month of ordinary spending painted red reads as a month of errors.

7.6 Edge cases and tests

CONDITION	BEHAVIOUR
No entries and no accounts	Empty state, not a screen of zeros
Expenses but no income	Savings rate blank; verdict reads "No income logged"
Pending transaction	Shown greyed, excluded from income, expenses and net
Pending charge settles	Upsert on <code>externalID</code> updates the row in place; no duplicate
Credit card with a balance	Subtracts from net worth
No accounts linked	Net worth is <code>nil</code> ; the card is not rendered at all
Entry added mid-month	<code>ModelContext.didSave</code> reloads every tab, not just this one

TESTS

`LifeOSKit/Tests/PersistenceTests/MoneyTests.swift` , all passing:

- `summaryUsesOurSignConventionNotPlaids` – the regression test for the sign flip.
- `savingsRateIsUndefinedWithoutIncome` – `nil` , never `0` .
- `pendingTransactionsAreExcludedFromTheRollup` .
- `creditAndLoanBalancesSubtractFromNetWorth` .
- `ingestUpdatesAPendingTransactionRatherThanDuplicatingIt` – the upsert, keyed on Plaid's `transaction_id` .

8 Plan

Goals, habits, notes and content behind one segmented pill. Four sections, four renderings, and underneath them a single row shape, because a dashboard you can rearrange later is a dashboard whose parts must be interchangeable now.

8.1 Intent

WORKING BACKWARDS · V1 FEATURE

Plan holds four lists. Goals step through milestones and show a progress bar. Habits tick once a day and carry a fourteen-day dot strip with a streak count. Notes are text. Content is a month calendar with a marker on every scheduled day and the items grouped by status beneath it.

Why is one model behind all four?

Because a goal, a habit, a note and a content item differ in which fields they use, not in shape. One model with a `kind` discriminator means a future block renderer can lay out any entry without knowing which feature produced it, and adding a fifth kind is a new enum case rather than a new table, a new store and a new screen.

Why does an unticked habit today not break the streak?

Because the day is not over. A streak that resets at 00:01 punishes you for not having done the thing yet, and a counter that lies about the present is one you stop trusting about the past.

Why is the journal not one of these sections?

It is the same model, kind `.journal`, but it surfaces in Body. How the day felt belongs next to how the body performed, not next to a content calendar.

8.2 Files and folders

```

Features/Plan/
├── Model/
│   └── PlanSnapshot.swift           the four sections, their hues and prompts
├── ViewModel/
│   └── PlanViewModel.swift         load, add, advance, toggle, delete
└── View/
    ├── PlanScreen.swift           pill, list rendering per section
    └── ContentCalendar.swift       month grid, status groups
  
```

LifeOSKit/Sources/Persistence/

└─ PlanEntry.swift
└─ PlanStore.swift

entry, tick, kind, status, snapshot type
queries, ticks, streaks

8.3 Model

```
public enum PlanKind: String, Codable, Sendable, CaseIterable {  
    case goal, habit, note, content, journal  
}  
  
public enum PlanStatus: String, Codable, Sendable, CaseIterable {  
    case todo, inProgress, done, blocked, scheduled  
}  
  
public protocol PlanItem: Sendable {  
    var id: UUID { get }  
    var kind: PlanKind { get }  
    var title: String { get }  
    var detail: String? { get }  
    var status: PlanStatus { get }  
    var sortOrder: Int { get }  
    var dueDate: Date? { get }  
    /// 0...1 when the entry tracks completion, nil when it does not.  
    var fraction: Double? { get }  
}
```

`PlanItemSnapshot` is the detached value form of that protocol and the only thing views ever see. It computes `fraction` once, in its initialiser, clamped to `0...1`, so no view has to divide by a target that might be zero.

RAW STRINGS, ON PURPOSE

`PlanEntry` stores `kindRaw` and `statusRaw` as `String` and exposes typed accessors over them. `SwiftData` would happily store the enums; storing the raw value means a new case is a code change rather than a migration, and an unrecognised value on the way back reads as `.note` or `.todo` instead of failing the fetch.

```
@Model  
public final class HabitTick {  
    public var entryID: UUID  
    public var date: Date           // always Calendar.startOfDay  
}
```

A habit's history is a set of days, not a field, so ticks are their own model. Keying by `entryID` rather than a `SwiftData` relationship keeps the tick table flat enough to query by date range, at the cost of one explicit

cleanup: `PlanStore.delete` removes an entry's ticks by hand, because an orphaned tick would resurrect a deleted habit's streak if the identifier were ever reused.

8.4 ViewModel

```
@MainActor @Observable
final class PlanViewModel {
    private(set) var snapshot = PlanSnapshot()
    var section: PlanSection = .goals

    func load()
    func add(title: String, detail: String?, target: Double?, dueDate: Date?)
    func advance(id: UUID)
    func toggleHabit(id: UUID)
    func delete(id: UUID)
}
```

`load()` fetches each kind separately and, for habits only, asks the store for a fourteen-day tick window and a streak per entry. That history is carried in the snapshot so the dot strip renders from a value rather than querying per row while the list scrolls.

`advance` carries the one piece of behaviour that differs by kind:

```
if let target = entry.progressTarget {
    entry.progressValue = min((entry.progressValue ?? 0) + 1, target)
    entry.status = (entry.progressValue ?? 0) >= target ? .done : .inProgress
} else {
    entry.status = entry.status == .done ? .todo : .done
}
```

A goal with milestones steps through them and completes itself on the last one. Everything else toggles. The four mutating methods share a `mutate(id:)` helper that fetches the entry, applies the change and reloads, so no view holds a live `PlanEntry`.

STREAKS

```
public func streak(for entry: PlanEntry, endingOn end: Date = .now) throws -> Int {
    let ticks = try recentTicks(for: entry, days: 365, endingOn: end)
    var count = 0
    for (index, done) in ticks.reversed().enumerated() {
        if done { count += 1 }
        else if index == 0 { continue } // today still open
        else { break }
    }
    return count
}
```

Walk backwards from today, counting until a gap. The `index == 0` clause is the whole rule from the PR FAQ, in one line: an unticked today is skipped rather than treated as a break. Yesterday unticked still ends the streak.

8.5 View

`PlanScreen` renders the pill and one list. Goals get a progress bar and a milestone count, habits get a dot strip and a streak, notes get their text, and content hands off to `ContentCalendar`.

`ContentCalendar` builds a day-to-statuses map once per render and reuses `MonthGridLayout` for the calendar arithmetic rather than repeating it. The leading-blank and month-boundary maths is already covered by `MonthGridLayoutTests`; a second copy of it inside a feature would not be.

Each `PlanSection` carries its own hue and its own add prompt, so the canvas tint and the sheet's placeholder both follow the selection without a switch in either view.

8.6 Edge cases and tests

CONDITION	BEHAVIOUR
Habit ticked twice in one day	Second tap removes the tick; one tick per day maximum
Today not yet ticked	Streak unchanged; the day is still open
Yesterday missed	Streak ends there
Habit deleted	Its ticks are deleted with it, explicitly
Goal with no target	<code>fraction</code> is <code>nil</code> ; no bar is drawn
Goal target reached	Clamped at the target; status becomes <code>.done</code>
Content item with no due date	Absent from the calendar grid, present in the status groups
Empty section	Per-section empty state carrying that section's add prompt

TESTS

`LifeOSKit/Tests/PersistenceTests/MoneyTests.swift` also holds `PlanStoreTests`, all passing:

- `toggleingAHabitIsIdempotentPerDay`.
- `recentTicksAreOldestFirstAndSpanTheWindow`.
- `anOpenTodayDoesNotBreakTheStreak`.
- `deletingAHabitRemovesItsTicks`.
- `entriesAreScopedToTheirKind` — the guard on the one-model decision.
- `goalFractionComesFromMilestones`.

9 Settings

Neutral. Where the goals that define a filled dot are edited, where every connection reports its real state, and where sync failure is allowed to be visible, because it is the only place in the app where it is allowed to be visible at all.

9.1 Intent

WORKING BACKWARDS · V1 FEATURE

Settings shows connection status for Health, Whoop, and Supabase, the last time each one delivered anything, and the five numbers that decide what a good day is. Change a goal here and the dot grid on Today re-reads itself against the new standard.

Why can I edit the goals at all?

Because the dot grid is the app's central claim, and a claim you can't calibrate is one you stop believing. The defaults are 8,000 steps, 7h sleep, 30 min exercise, 2,500 ml water, 3 of 4 required.

What does Settings do when HealthKit access was denied?

It cannot know. HealthKit read authorization is opaque by design. Settings therefore shows the last time data arrived and links to the Health app's own permission screen, rather than asserting a state it cannot verify.

Where does the quick log live?

Not here. It is a sheet on the root FAB, reachable from any tab, because logging water is the one thing you do repeatedly and it must never be four taps deep.

9.2 Files and folders

```

Features/Settings/
├── Model/
│   ├── SettingsSnapshot.swift    connections + last-sync times
│   └── GoalDraft.swift           editable copy of GoalTargets
├── ViewModel/
│   └── SettingsViewModel.swift    edit, validate, commit
└── View/
    ├── SettingsScreen.swift
    └── GoalEditorSection.swift    five steppers, one per target

Components/View/QuickLogSheet.swift ← water + weight, presented from the FAB
Components/ViewModel/QuickLogViewModel.swift

```

9.3 Model

```
struct SettingsSnapshot: Equatable, Sendable {
    var health:    ConnectionStatus
    var whoop:     ConnectionStatus
    var supabase:  ConnectionStatus
}

struct GoalDraft: Equatable, Sendable {
    var steps: Int           // 1,000-30,000
    var sleepMinutes: Int    // 240-660
    var exerciseMinutes: Int // 5-180
    var waterML: Double      // 500-6,000
    var requiredCount: Int    // 1-4

    var targets: GoalTargets { ... }
}
```

The draft is a separate type from `GoalTargets` on purpose. Editing writes to the draft; committing validates it into targets. Binding steppers straight to the persisted model would write to the database on every increment and re-evaluate a month of history mid-drag.

9.4 ViewModel

```
@MainActor @Observable
final class SettingsViewModel {
    /// Bound directly by the form's steppers. Writes are committed by save().
    var draft = GoalsDraft()

    /// Derived, not stored.
    var connections: [ConnectionStatus] { ... }

    func attach(_ context: ModelContext)
    func load()
    func save()
}
```

`draft` is the one mutable published property on any view model in this app, and it is mutable because SwiftUI's `@Bindable` needs it to be. Everything else stays `private(set)`: the View proposes, the ViewModel disposes.

CONNECTION STATUS IS DERIVED ON EVERY READ

The Supabase row used to be a hardcoded "Not configured", which became a lie the moment the project was linked. It now reads `AppConfig.supabaseURL` each time. Whoop's live state comes from `WhoopConnectionViewModel`, which `AppShell` owns and shares with onboarding, so a connection made in either place is immediately true in the other.

9.5 View

```
struct SettingsScreen: View {
  @Bindable var model: SettingsViewModel
  @Bindable var whoop: WhoopConnectionViewModel

  var body: some View {
    Form {
      Section("Connections") { /* one row per model.connections, plus Whoop */ }
      Section("Daily goals") { /* steppers bound to $model.draft */ }
    }
    .onAppear { model.load() }
    .onDisappear { model.save() }
  }
}
```

Settings is presented as a sheet from the gear on Today rather than owning a tab. Goals are committed on dismissal, in one write, so dragging a stepper does not re-evaluate a month of history on every increment.

QUICK LOG

`QuickLogSheet` is a component rather than a feature because it is presented from the root, over whichever tab you happen to be on. Its view model holds the two draft values and calls `MetricsWriting.logWater / logWeight`, which upsert into today's row. It is the only writing path in V1 that originates from a person rather than a device.

9.6 Edge cases and tests

CONDITION	BEHAVIOUR
Goal set out of range	Clamped at commit; the stepper ranges make it unreachable in the UI anyway
Goal of zero	Excluded from every range, because a zero goal makes progress ratios meaningless
HealthKit denied	Undetectable. Shows last-received timestamp and a link out to Health
Sync stalled	Stale last-sync timestamp. Never a blocking error, never an alert
Whoop token cleared	<code>.needsReauth</code> ; the reconnect action restarts the PKCE flow
Quick log while offline	Committed locally; <code>syncedAt</code> stays <code>nil</code> until the next successful push

TESTS

- `SettingsViewModel`: out-of-range drafts clamp; committing writes exactly once.
- `QuickLogViewModel`: logging water twice in a day accumulates into one row rather than creating two.

10 The Coach

One card every morning: what you did well yesterday, what slipped, and one thing to fix today. Built from your actual numbers, on-device, and structurally unable to invent them.

10.1 Intent

WORKING BACKWARDS · V2 FEATURE

Every morning, one card: what you did well yesterday, what slipped, and one thing to fix today. Built from your actual weight, sleep, recovery, steps, calories, and water. Ask it anything, "why is my recovery tanking?", and it queries your real records to answer rather than guessing.

Does my health data go to a server?

Usually not. The coach floors on Apple's on-device Foundation Models: free, private, offline, no API key. Some requests genuinely exceed what a small model can do, and those escalate to a cloud model. What escalates is a `MetricsDigest`: derived numbers only, never raw Whoop payloads, heart-rate series, or identifiers. Food photos always leave, because the on-device model is text-only and cannot see an image.

So what actually leaves the device, and when?

Only on an escalated request, and only aggregates. The privacy rule is enforced by a type rather than by review: `Engine.run` takes a `MetricsDigest`, and `DailyMetrics` is a `SwiftData @Model` class that is not `Sendable`, so the compiler prevents a raw row from crossing the boundary. The calendar assistant is the one exception, and it is called out in 15.2: event titles are content, not numbers, and cannot be aggregated without destroying their use.

How does it avoid making numbers up?

Tool calling. The model doesn't receive your data in a prompt; it calls Swift functions that query `DailyMetrics` and returns what they hand back. A number it had to look up is a number it cannot hallucinate, and the context stays small enough for a 3B on-device model.

What if the on-device model isn't available?

`SystemLanguageModel.availability` is checked at runtime, and the request escalates to the cloud tier. Three reasons are distinguished: `.deviceNotEligible` is permanent and resolved once at launch, `.appleIntelligenceNotEnabled` is user-fixable and re-checked on foreground, and `.modelNotReady` is transient while weights download and is re-checked per request.

Why is the floor iOS 26?

Because the coach floors on `FoundationModels`, which does not exist before it. The TestFlight floor was briefly 18.0; that is reversed. Devices below iOS 26 cannot install the app, so there is no older-OS cohort that always escalates. Unavailability is only the three runtime reasons above, and the projections in chapter fifteen stay per escalated turn because capability — not OS version — is what still sends work to the cloud.

10.2 Files and folders

Features/Coach/	
└─ Model/	
└─ CoachBrief.swift	the morning card
└─ ChatTurn.swift	one message, role + text + state
└─ CoachAvailability.swift	available · downloading · unsupported
└─ ViewModel/	
└─ CoachViewModel.swift	session lifecycle, streaming, tool wiring
└─ View/	
└─ CoachScreen.swift	
└─ BriefCard.swift	
└─ ChatTranscript.swift	
LifeOSKit/Sources/Insights/	the inference layer, reads Persistence only
└─ CoachTask.swift	the contract every task conforms to
└─ CoachTool.swift	the tool contract, shared by both tiers
└─ ToolLoop.swift	provider-neutral round loop
└─ CoachRouter.swift	tier selection and escalation
└─ Tasks/	DailyBrief · WeeklyReview · CoachAnswer · MealEstimate
└─ Engines/	Engine · OnDeviceEngine · RemoteEngine
└─ Context/MetricsDigest.swift	Persistence to aggregates; the privacy boundary
└─ SchemaJSON.swift	GenerationSchema to JSON Schema

WHY THE TOOLS LIVE IN THE PACKAGE, NOT THE FEATURE

The functions the model calls are domain logic with a strict dependency rule: `Insights` may read `Persistence` and nothing else. Putting them in `Features/Coach/` would let a future edit reach for a network client mid-conversation. The feature folder holds the conversation; the package holds what can be asked.

10.3 Model

```
struct CoachBrief: Equatable, Sendable {
    var date: Date
    var wentWell: [String]
    var slipped: [String]
    var oneThing: String
}

struct ChatTurn: Identifiable, Equatable, Sendable {
    enum Role { case you, coach }
    var id: UUID
    var role: Role
    var text: String
    var isStreaming: Bool
}

enum CoachAvailability: Equatable, Sendable {
    case available
    case downloading
    case unsupported(reason: String)
}
```

10.4 ViewModel

```
@Observable @MainActor
final class CoachViewModel {
    private(set) var availability: CoachAvailability = .downloading
    private(set) var brief: CoachBrief?
    private(set) var transcript: [ChatTurn] = []
    var input: String = ""

    private let session: InsightSession // wraps LanguageModelSession

    func start() async // checks availability first
    func generateBrief() async
    func send() async // appends, streams, replaces
}
```

Availability is checked before anything else happens, and a failed check is a rendered state rather than a thrown error. The chat streams into the last `ChatTurn` by mutating `text` and flipping `isStreaming`, so the View re-renders one row, not the transcript.

10.5 View

```
switch vm.availability {
case .available:
  ScrollView {
    if let brief = vm.brief { BriefCard(brief: brief) }
    ChatTranscript(turns: vm.transcript)
  }
case .downloading:
  CoachUnavailable(title: "Apple Intelligence is still downloading")
case .unsupported(let reason):
  CoachUnavailable(title: reason)
}
```

10.6 Edge cases and tests

CONDITION	BEHAVIOUR
Device lacks Apple Intelligence	Unavailable state with the reason shown. No cloud fallback without asking
Model still downloading	Reported as downloading with the reason surfaced, not a spinner
Tool returns no rows	Model is told "no data for that range" explicitly, so it says so instead of inventing a trend
Guardrail refusal	Surfaced plainly. Never retried with a reworded prompt to evade it
User asks about a future date	Tool returns empty; no speculation

TESTS

- The tools are tested in `InsightsTests` like any other pure function: given a fixture month, each returns exactly the rows it claims to.
- `CoachViewModel` with a stub session: each availability case renders its state; a streamed response mutates one turn rather than appending several.

11 Integrations

Four external systems and one that had to be cut. Each is confined to its own package target, so none of the detail below is visible from any screen, because a ViewModel sees only rows in `DailyMetrics` and a connection status.

11.1 Apple HealthKit

The primary source. Read-only in V1, so the app requests no write permissions, which keeps the authorization surface minimal and means Life OS can never corrupt your Health data.

TYPES READ

METRIC	IDENTIFIER	QUERY
Weight	<code>bodyMass</code>	Anchored
Steps	<code>stepCount</code>	Statistics collection
Active energy	<code>activeEnergyBurned</code>	Statistics collection
Exercise minutes	<code>appleExerciseTime</code>	Statistics collection
Resting HR	<code>restingHeartRate</code>	Anchored
HRV	<code>heartRateVariabilitySDNN</code>	Anchored
Water	<code>dietaryWater</code>	Statistics collection
Sleep	<code>sleepAnalysis</code> (category)	Anchored
Workouts	<code>HKWorkoutType</code>	Anchored

FRESHNESS

- `HKAnchoredObjectQuery` with a **persisted anchor per type**, so each sync fetches only what changed since last time.
- `enableBackgroundDelivery` so iOS wakes the app when new samples land.
- `HKStatisticsCollectionQuery` for daily aggregates, which de-duplicates across sources. Summing raw samples yourself double-counts steps on any day you carried both an iPhone and a Watch.

THE AUTHORIZATION TRAP

HealthKit read authorization is **deliberately opaque**. The API will not tell you whether the user denied a type, only whether you have asked. Branching on read authorization status is therefore always a bug. Query anyway, and treat an empty result as "no data," rendering an explicit empty state. Apple designed it this way so apps cannot detect that you are hiding a condition from them.

11.2 Whoop

The only source of Recovery %, Strain, and sleep staging, which is why it gets a direct API integration when every other wearable arrives through HealthKit. Everything below is **API v2** against base URL

`https://api.prod.whoop.com/developer/v2` ; v1 is deprecated, and v2 identifies sleep by UUID rather than integer ID. The whole integration lives in `LifeOSKit/Sources/Integrations` and runs **on the device**: `WhoopClient` fetches, `WhoopSync` orchestrates, every payload is archived raw into `SwiftData`, and `WhoopDerivation` turns the archive into the same `DailyMetrics` rows every screen already reads. No screen knows Whoop exists.

COLLECTIONS AND SCOPES, DEFINED IN ONE PLACE

`WhoopCollection` is a five-case enum pairing each endpoint *path* with the scope that authorizes it, and both the client and the OAuth request are driven from it:

COLLECTION	ENDPOINT	SCOPE	FEEDS
Recovery	GET <code>/recovery</code>	<code>read:recovery</code>	Recovery %, HRV, resting HR, SpO2, skin temp
Sleep	GET <code>/activity/sleep</code>	<code>read:sleep</code>	Duration, stages, performance / efficiency / consistency, sleep debt
Cycle	GET <code>/cycle</code>	<code>read:cycles</code>	Day strain, calories, average and max HR
Workout	GET <code>/activity/workout</code>	<code>read:workout</code>	<code>WorkoutRecord</code> rows, exercise minutes
Body	GET <code>/user/measurement/body</code>	<code>read:body_measurement</code>	Weight, while HealthKit is unconnected

The scope string sent to Whoop is exactly `WhoopCollection.allCases.map(\.scope) + ["offline"]`. `offline` is what makes Whoop return a refresh token; without it the connection dies the first time the access token expires, about an hour in. `read:profile` is deliberately *not* requested and `/user/profile/basic` never called: name and email are identity, not signal.

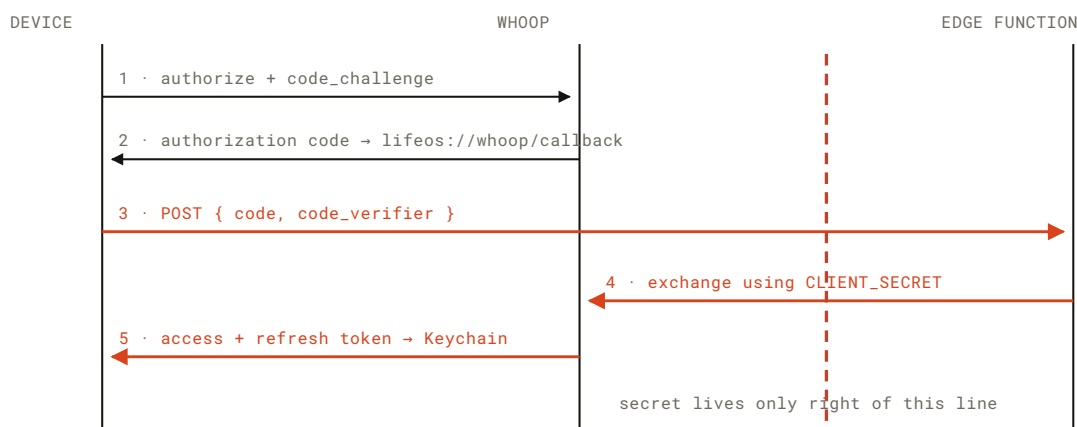
WHY PATH AND SCOPE LIVE IN THE SAME ENUM

They used to be listed separately, and drifted: workout and body were added to the client while the authorize request still asked for three scopes. Whoop then 401s a token it issued seconds earlier, and the sync read that 401 as a dead credential and deleted it, so connect → sync became an unbreakable loop. Colocating them makes a fetchable-but-unauthorized collection unrepresentable. The same incident is why a 401 from a *collection* endpoint no longer clears tokens (see below).

OAuth, AND WHY THE SECRET NEVER SHIPS

Whoop registers apps as *confidential* clients: the token exchange requires a client secret, and an `.ipa` is a zip file, so a secret in the binary is a secret published. The `whoop-token` Supabase Edge Function is the

fix: the app runs the authorize step with PKCE (`S256` challenge over a random 32-byte verifier) and obtains a `code`; the function holds `WHOOOP_CLIENT_SECRET` and performs the exchange. PKCE is kept even though the exchange is server-side, because it binds the code to this app instance — a code intercepted in transit is useless without the verifier.



Steps 3-5 are the whole point: the client secret is used at step 4, on the server, and never crosses back onto the device. Without this, the integration would be unshippable to anyone but its author.

Two implementation details of the authorize step matter in practice. First, the redirect is **two hops**: Whoop requires an https redirect URI, so the registered URI points at the `whoop-callback` Edge Function, which immediately 302s to `lifeos://whoop-callback` with the code and state attached. The hop holds no secret and makes no decisions; PKCE and the state check both still happen in the app, so anyone who reaches the endpoint gets a redirect back to an app that rejects a state it did not issue. Second, sign-in opens in **Safari via** `UIApplication.open`, not `ASWebAuthenticationSession`: Whoop's login page sits behind a Cloudflare bot challenge that never clears inside the auth-session web view — it renders blank on device and simulator alike — while real Safari passes it.

Because Safari backgrounds the app and iOS may terminate it before the callback returns, the pending PKCE verifier and state are persisted to the Keychain rather than held in memory, and as a *list* (capped at five, fresh for fifteen minutes): tapping Connect twice creates two valid in-flight authorizations, and the callback matches whichever state it carries. A separate manual flow exists for signing in on another device: state is prefixed `manual-`, the callback function renders the code on an HTML page for transcription instead of redirecting, and the user pastes it into Settings.

TOKEN LIFECYCLE

Tokens live in the Keychain (`kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly` , so never in a plist and never in iCloud backups) as `{accessToken, refreshToken, expiresAt}` . Refresh is **proactive**: `WhoopSync` treats a token as expired sixty seconds early and refreshes through the same Edge Function *before* fetching, rather than reacting to a 401 mid-sync. Whoop rotates the refresh token on every use, so the new pair is persisted before anything else happens.

ONLY A REFRESH 401 CLEARS THE CREDENTIAL

A 401 from the *refresh* call means the grant itself is dead (revoked in Whoop's app, or the rotated token was lost), so tokens are cleared and Settings shows reconnect. A 401 from a *collection* endpoint throws `accessDenied` and keeps the tokens: it usually means the token's scopes predate the current scope list, and the fix is disconnect-and-reconnect, which the error message says in those words. Treating the two alike is what once deleted a credential granted seconds earlier.

FETCHING: PAGINATION, DATES, AND SCORING STATES

- **Paged collections** take `start / end` (plain ISO 8601, no fractional seconds) and `limit=25`, which is Whoop's maximum page size, not a choice. The response's `next_token` is passed back as `nextToken` until exhausted, with a cap of 20 pages per collection per sync; hitting the cap logs an error to OSLog and returns what it has.
- **Date decoding is custom.** Whoop timestamps carry fractional seconds (`2026-08-10T06:16:16.180Z`), which Foundation's stock `.iso8601` strategy rejects — with the default decoder, every record fails. The decoder tries fractional-seconds first, then plain internet date-time.
- **Unscored records are archived but not derived.** Records arrive with a `score_state` of `SCORED`, `PENDING_SCORE`, or `UNSCORABLE`; only scored ones become samples. They still land in the raw archive, so a record that scores later is picked up by the next sync's re-derive.
- **No retry logic.** A 429 surfaces immediately as "Rate limited by Whoop"; there is no backoff loop to write until something automated calls this code (see below).

THE SYNC PIPELINE: ARCHIVE FIRST, DERIVE SECOND

`WhoopSync.sync()` is the whole ingestion story, and its order is the design:

1. Load tokens (throw `notConnected` if absent), refresh proactively if expired.
2. Fetch all five collections for a **rolling trailing 14-day window**, sequentially.
3. **Archive every raw payload** into `WhoopRawRecord`, split per record, before any interpretation. A payload that was never stored cannot be re-derived from.
4. Run `WhoopDerivation` over the fetched records to upsert `DailyMetrics`, `SleepRecord`, and `WorkoutRecord` rows.

There is no incremental cursor and no watermark: every sync re-fetches the same fourteen days, and idempotency comes entirely from upserts — the archive on `(kind, externalID)`, `DailyMetrics` on the day, sleep and workout records on their provider IDs. Re-syncing is therefore always safe, and "which records are new" is a question that never needs answering. Sync runs on exactly three triggers, all user-visible: a successful connect, a successful manual code exchange, and the "Sync now" button in Settings. There is no background task and no timer yet; the nightly server-side job and the `whoop-webhook` receiver from 11.3 remain planned, and are what will eventually make morning data exist before the app opens.

THE RAW ARCHIVE, AND WHY IT EXISTS

`WhoopRawRecord` stores `{kind, externalID, payload, receivedAt}`, unique on `(kind, externalID)`. `kind` is a plain string rather than an enum so an unseen collection lands as data, not as a failed insert. Two facts justify keeping every byte: Whoop rate-limits, and a derivation bug found six months from now must be fixable without asking for the data again. `WhoopDerivation.rederive()` rebuilds every derived row from the archive with **zero network calls** — and because every normal sync also derives from freshly archived payloads, the archive path is exercised on every sync and cannot rot. The columns deliberately mirror the Supabase `whoop_raw` table, so moving ingestion server-side later is a copy, not a redesign.

ONE SPLITTING QUIRK WORTH KNOWING

Recovery records carry no `id` of their own — a recovery is keyed by the *cycle* it scores, so `WhoopRawSplit` resolves `id` first and falls back to `cycle_id`. And because splitting re-serialises each record through `JSONSerialization`, the archive is semantically faithful but not byte-faithful (key order and number formatting may differ), a known and accepted limitation.

DERIVATION: FROM PAYLOADS TO DAILYMETRICS

`WhoopDerivation` holds two invariants. Every write goes through `MetricsStore.upsertBatch`, which merges into the existing row for that day rather than replacing it — HealthKit owns `steps` on the same rows. And a `nil` field **never overwrites a stored value**: a sample that omits a field means "no reading," not "clear what you had."

SOURCE	ATTRIBUTED TO	WRITES
Recovery	Day of <code>created_at</code>	<code>whoopRecoveryPct</code> , <code>restingHR</code> , <code>hrvMs</code> , <code>spo2Percentage</code> , <code>skinTempCelsius</code>
Sleep	Day the sleep ends — a night belongs to the morning you woke up	<code>sleepMinutes</code> , <code>respiratoryRate</code> , Whoop sleep performance / efficiency / consistency / debt, plus a full <code>SleepRecord</code> with stage minutes
Cycle	Day the cycle starts	<code>whoopDayStrain</code> , <code>whoopCalories</code> , <code>whoopAverageHR</code> , <code>whoopMaxHR</code>
Workout	Day of workout start	<code>exerciseMinutes</code> , plus a full <code>WorkoutRecord</code> (kcal, strain, HRs, distance, altitude, sport)
Body	Sync day (payload carries no date)	<code>weightKg</code>

Units are converted at the boundary, once: Whoop reports energy in kilojoules (`kilojoule / 4.184` → kcal, the divisor is exact) and durations in milliseconds (`milli / 60_000` → minutes). Asleep time is the sum of light + REM + slow-wave stages, preferred over `in_bed - awake` because that difference drifts whenever Whoop counts time as neither. Naps are archived and stored as `SleepRecord`s but excluded from the day roll-up, so an afternoon nap never inflates a night's sleep number. Constants are not ingested: height and body-measurement max HR would restate the same value on every row.

The DTO→sample mapping lives exactly once, as computed properties on the DTOs, because `rederive()` and the live sync must agree by construction — a duplicated copy of that mapping is the

kind of thing that drifts silently. Open follow-ups from the ingestion reviews (the "Synced N days" count over-counts, `user_calibrating` is parsed then dropped, a page-cap truncation is visible only in OSLog) are recorded in `docs/superpowers/plans/2026-08-12-whoop-full-ingestion-followups.md` and remain open.

WHAT THE SCREENS SEE

Nothing above. `RecoveryViewModel` reads a 14-day window of `DailyMetrics` and `SleepRecord` rows from `SwiftData` — the window deliberately matches the sync window, so a chart never has a gap the sync could have filled — and the Recovery screen and `WhoopDetailScreen` render trends where a missing day is a visible gap, never a zero-height bar. The only Whoop-aware surface in the app is the connection row in Settings, driven by `WhoopConnectionViewModel`'s five states: unconfigured, disconnected, connecting, connected ("Synced N days"), failed — with failure messages written to distinguish "get a fresh code" from "reconnect to grant new scopes," because those need opposite fixes.

11.3 Supabase

SCHEMA

Mirrors the local `SwiftData` models: `daily_metrics`, `workout_records`, `sleep_records`, `whoop_raw`, and `sync_state`.

`whoop_raw` stores unprocessed API payloads. This is deliberate: when Whoop's response shape differs from the documentation, and it will, you can re-derive every downstream value without re-fetching a month of history against a rate limit. Today the live archive is the local `WhoopRawRecord` model (11.2), whose columns deliberately mirror this table; uploading it is deferred until ingestion moves server-side, at which point the move is a copy, not a redesign.

ROW LEVEL SECURITY

RLS IS ENABLED ON EVERY TABLE, WITHOUT EXCEPTION

Policies scope all access to `auth.uid()`. This is precisely what makes shipping the anon key inside the app safe. An RLS misconfiguration does not throw. It returns an *empty result set*, which looks exactly like "no data yet." Verify policies with a real query before trusting an empty screen.

AUTH

`SupabaseAuth` talks to `auth/v1` over REST with no SDK. The five calls the app makes (ask which channels are enabled, request a code or link, verify, refresh, update the user record) are small enough that a dependency would pull in far more surface than it removes. The anon key goes in the `apikey` header; it is public by design, and RLS is what protects rows.

ENDPOINT	USED FOR	NOTE
GET auth/v1/settings	Which channels the project can deliver on	Falls back to email-only if the call fails
POST auth/v1/otp	Six-digit code (phone) or magic link (email)	<code>create_user: true</code> ; this is the signup path
POST auth/v1/verify	Exchange the code for a session	Phone only, while email uses the link
POST auth/v1/token	Renew an expired session	<code>grant_type=refresh_token</code> . Rotates the token; an invalid one answers 400
PUT auth/v1/user	Name and country into <code>user_metadata</code>	Avoids a profiles table before there is anything else to put in one

THE MAGIC-LINK SESSION ARRIVES IN THE FRAGMENT

Supabase puts `access_token` in the URL *fragment*, not the query.

`URLComponents.queryItems` returns nothing, so the callback reads as empty and the user appears to have signed in to nothing. `AuthSession(callback:)` parses both fragment and query by hand. The same is true of failures, which arrive as `error_description` in the fragment.

The session is held in the Keychain by `KeychainAuthSessionStore` , which is what lets `AppShell` skip signup on relaunch. Email currently signs in by magic link rather than a code because the free tier will not let the email template carry a token; see 4.4.

SESSION LIFECYCLE

An access token lasts an hour, so the Keychain alone only keeps a user signed in until the token behind the session goes stale. `SessionRefresher` is the piece that makes "stay logged in" true: `AppShell` calls it on launch and on every return to the foreground, and it resolves to one of three outcomes — `signedOut` (nothing stored), `active` (a usable session, untouched or just renewed), or `rejected` (the server refused the credential, and the store has been cleared).

ONLY A REFUSAL ENDS A SESSION

Signing someone out cannot be undone without a fresh code, so it happens only when the server has definitively refused the refresh token — 400 , 401 , 403 or 422 . A transport failure is not an answer, and 429 and every 5xx are temporary by definition, so all of them keep the session and try again later. Opening the app on a plane must not log you out.

Two details are load-bearing, and both are there because the alternative fails quietly. Supabase **rotates the refresh token on every use**, exactly as Whoop does (11.2), so concurrent refreshes are not merely wasteful: the second presents a token the first has already retired and is refused, which would sign a user out for doing nothing but foregrounding the app while it launched. `OnboardingViewModel` therefore coalesces callers onto a single in-flight `Task` . And a refreshed session **inherits what the old one knew**: a

response that omits the user, or the magic-link path that never had a user id to begin with, must not erase an identity already on file, nor drop a refresh token and leave the next launch with nothing to present.

EDGE FUNCTIONS

FUNCTION	STATUS	PURPOSE	SECRETS USED
whoop-token	Deployed	Exchange auth code / refresh token	WHOOP_CLIENT_SECRET
whoop-callback	Deployed	https → lifeos:// redirect bridge; renders the code for transcription when signing in on another device	none
whoop-sync	Planned	Scheduled nightly pull	WHOOP_CLIENT_SECRET , SERVICE_ROLE_KEY
whoop-webhook	Planned	Receive Whoop push events	WHOOP_WEBHOOK_SECRET , SERVICE_ROLE_KEY
coach V2	Planned	Role to model mapping, price lookup, spend reservation, and the cloud inference call for every escalated request including meal photos and the calendar assistant	OPENROUTER_API_KEY , COACH_MODEL_REASONING , COACH_MODEL_VISION , SERVICE_ROLE_KEY

whoop-callback holds no secret and makes no decisions. PKCE and the state check both still happen in the app, so the hop cannot be used to inject a forged code: anyone who reaches the endpoint gets a redirect back to an app that rejects a state it did not issue.

11.4 Foundation Models

The coach floors on Apple's on-device model and escalates when it cannot serve.

CORRECTION TO REVISION C

Revision C stated that all tiers sit behind the same `LanguageModelSession` API, so routing is configuration rather than a rewrite. **That is not true on iOS 26.** Verified against the iOS 26.2 SDK interface: `SystemLanguageModel` is a `final class`, there is no public `LanguageModel` or `LanguageModelExecutor` protocol, and `LanguageModelSession` exposes no seam for a non-Apple backend. Those protocols arrive in iOS 27. Until then a thin abstraction of our own is required, not optional. The goal is to keep it small enough that adopting Apple's seam later deletes `RemoteEngine` rather than rewriting its callers.

TIER	USED FOR	CONTEXT	COST	AVAILABLE
SystemLanguageModel	Daily brief, chat, calendar assistant	Small	Free, private, offline	iOS 26
OpenRouter, role reasoning	Weekly review; anything escalated	1M	See ch. 15	Any
OpenRouter, role vision	Food photo → macros	Large	See ch. 15	Any
PrivateCloudComputeLanguageModel	A third tier: key-free and private	32K	Free, private	iOS 27

The app never names a model. It names a `Tier.Role`, and the proxy maps role to model from `COACH_MODEL_REASONING` and `COACH_MODEL_VISION`. Upgrading the coach is then `supabase secrets set`: no deploy, no client change, no App Store review. The moment a model ID appears in Swift, that becomes a shipping event, which is the whole return on keeping the proxy thin.

When iOS 27 ships the public `LanguageModel` and `LanguageModelExecutor` protocols, `RemoteEngine` conforms to Apple's protocol and `CoachRouter` collapses. Callers do not change, which is the point of keeping our own abstraction thin rather than absent.

ONE INFERENCE LAYER, NOT ONE PER FEATURE

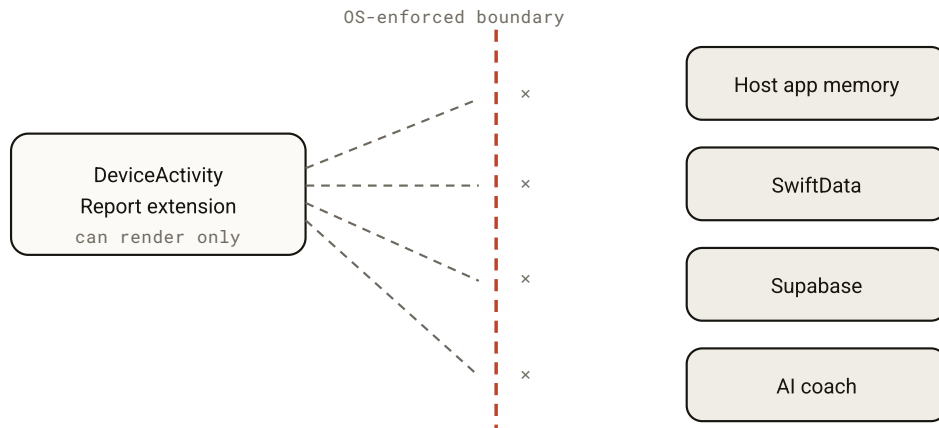
Everything that needs a model goes through `Insights`: the daily brief, chat, the weekly review, meal photos, and the calendar assistant. One proxy, one key, one role map, and one spend ceiling. A feature that stood up its own proxy would also stand up a second budget, and two budgets cannot jointly honour one ceiling.

TOOL CALLING IS THE ANTI-HALLUCINATION MECHANISM

The model is not handed your data in a prompt. It calls Swift functions that query `DailyMetrics`. Two consequences: a number it had to look up is a number it cannot invent, and the context stays small enough for an on-device model to handle a month of history.

11.5 Screen Time

Screen time was an explicit product requirement. It cannot be built as specified, and the reason is worth documenting precisely so it is not re-attempted.



Usage figures can be displayed inside the report extension and nowhere else. The host app cannot read them into memory, persist them, transmit them, or pass them to the coach.

- **iPhone / iPad:** a screen-time *view* is possible; screen-time *analysis alongside health data* is not. It would also require Apple to grant the Family Controls distribution entitlement.
- **MacBook:** no API exists at any privilege level. Manual entry only, if ever.

Since the entire product thesis is the join across domains, a signal that provably cannot enter the join has no home here. Cut rather than half-built.

12 Data reference

The types every ViewModel in Chapters 4 to 9 reads through. Everything here lives in `LifeOSKit/Sources/Persistence` and depends on nothing.

12.1 DailyMetrics

One row per calendar day. The join key for the entire application.

PROPERTY	TYPE	WRITTEN BY	NOTES
<code>date</code>	<code>Date</code>	—	Always <code>startOfDay</code> . Unique. Never a timestamp.
<code>weightKg</code>	<code>Double?</code>	HealthKit, Quick log	Renpho → Health → here
<code>steps</code>	<code>Int?</code>	HealthKit	De-duplicated across sources
<code>activeEnergyKcal</code>	<code>Double?</code>	HealthKit	
<code>exerciseMinutes</code>	<code>Int?</code>	HealthKit	
<code>sleepMinutes</code>	<code>Int?</code>	HealthKit	Attributed to the wake day
<code>waterML</code>	<code>Double?</code>	Quick log	No automatic source exists
<code>restingHR</code>	<code>Double?</code>	HealthKit	
<code>hrvMs</code>	<code>Double?</code>	HealthKit	SDNN
<code>whoopRecoveryPct</code>	<code>Double?</code>	WhoopKit	Exists nowhere else
<code>whoopDayStrain</code>	<code>Double?</code>	WhoopKit	Exists nowhere else
<code>whoopSleepPerformancePct</code>	<code>Double?</code>	WhoopKit	
<code>updatedAt</code>	<code>Date</code>	MetricsStore	Drives field-level LWW
<code>syncedAt</code>	<code>Date?</code>	SyncKit	<code>nil</code> = never synced

EVERY METRIC IS OPTIONAL, AND THAT IS NOT NEGOTIABLE

A missing value and a zero must never be representable as the same thing. Zero steps means you didn't move; `nil` steps means the watch was on the charger. In a health app a false zero is worse than a blank. It is a wrong number presented with total confidence. This is why the optional is carried unchanged from the column through the snapshot to the tile, which renders `value ?? "—"` itself.

12.2 UserGoals

Singleton by convention: exactly one row, created with defaults on first access.

PROPERTY	TYPE	DEFAULT	RANGE IN SETTINGS
stepsGoal	Int	8,000	1,000-30,000
sleepMinutesGoal	Int	420 (7h)	240-660
exerciseMinutesGoal	Int	30	5-180
waterMLGoal	Double	2,500	500-6,000
requiredCount	Int	3	1-4

12.3 Source records

`WorkoutRecord` and `SleepRecord` hold data that is genuinely not daily. They roll *up* into `DailyMetrics`, which means the daily row is always safe to recompute and never the only copy of anything.

12.4 Goal evaluation

```
public func evaluate(_ reading: DayReading,
                    against targets: GoalTargets) -> DayStatus
```

Returns `.onTarget`, `.missed`, or `.noData`.

INPUT	RESULT	RATIONALE
No metrics present at all	<code>.noData</code>	Not a failure; excluded from streaks
$\geq$ <code>requiredCount</code> goals met	<code>.onTarget</code>	Default 3 of 4
Some data, below required count	<code>.missed</code>	An unlogged metric cannot launder a bad day
Exactly at a goal value	Counts as met	<code>>=</code> , not <code>></code>

```
public func currentStreak(statuses: [DayStatus]) -> Int
```

Expects most-recent-first ordering. `.noData` days are transparent, so they neither extend nor break a streak. `.missed` terminates immediately.

`DayStatus` is deliberately separate from `DesignSystem.DotState`, because `Persistence` must not depend on `DesignSystem`. The mapping between them happens in a feature `ViewModel`; see 4.4.

12.5 Store protocols

`MetricsStore` is the only path by which anything writes to `DailyMetrics`. It enforces the one-row-per-day invariant so callers cannot get it wrong. This is what ships today:

```
public struct MetricsStore {
    public init(context: ModelContext, calendar: Calendar = .current)

    @discardableResult
    public func upsert(date: Date, apply: (DailyMetrics) -> Void) throws -> DailyMetrics
    public func upsertBatch(dates: [Date], apply: (Date, DailyMetrics) -> Void) throws
    public func metrics(from: Date, to: Date) throws -> [DailyMetrics] // most-recent-first
    public func goals() throws -> UserGoals // defaults on first ac
}
```

`upsert` normalises any timestamp to `startOfDay` and **merges**: fields the closure doesn't touch are preserved. That is what lets HealthKit and Whoop write the same row safely.

THE PROTOCOLS MVVM NEEDS TO BE ADDED

The concrete API above returns `@Model` instances, which a ViewModel must not hold, see 2.6. Two protocols close that gap, implemented by `MetricsStore` in an extension:

```
public protocol MetricsReading: Sendable {
    func days(from: Date, to: Date) async throws -> [DayRecord] // most-recent-first
    func targets() async throws -> GoalTargets
}

public protocol MetricsWriting: Sendable {
    func logWater(_ millilitres: Double, on date: Date) async throws
    func logWeight(_ kilograms: Double, on date: Date) async throws
    func updateTargets(_ targets: GoalTargets) async throws
}
```

`DayRecord` is a `Sendable` struct mirroring the columns of `DailyMetrics`. The store converts on the way out.

WHY THE PROTOCOLS RETURN VALUE TYPES

Handing a `@Model` instance to a ViewModel would give every screen a live, mutable handle on the database and make each one require a `ModelContainer` to test. Returning `DayRecord` costs one map per fetch and buys back both.

The protocol methods are `async` while the store's are synchronous. That is deliberate: the store wraps trivially today, and when a fetch eventually moves off the main actor no ViewModel or test has to change.

12.6 Money and plan models

These sit beside the daily spine rather than on it. A transaction and a habit are both dated, but neither is a daily reading, and forcing them into `DailyMetrics` columns would mean one row per day could hold only one of each.

MODEL	KEY FIELDS	INVARIANT
MoneyEntry	date , amount , merchant , category , externalID , pending	date is startOfDay ; positive is money in ; externalID is Plaid's transaction_id and is the upsert key
MoneyAccount	name , type , currentBalance	credit and loan types contribute - abs(balance) to net worth
PlanEntry	kindRaw , title , statusRaw , sortOrder , dueDate , progressValue , progressTarget	Enums stored raw so a new case is not a migration; an unknown value reads as .note / .todo
HabitTick	entryID , date	date is startOfDay ; at most one per entry per day; deleted explicitly with its entry

One model carries five kinds. `PlanKind` is `goal` , `habit` , `note` , `content` or `journal` , and the journal case is why the Body tab writes through `PlanStore` . Views never see any of these types: both stores hand back `PlanItemSnapshot` and `MoneyRow` value types, for the reason given in the note above.

NOT YET ON THE SYNC SCHEMA

`supabase/migrations` covers `daily_metrics` , `workout_records` , `sleep_records` , `whoop_raw` and `sync_state` . The four models above are local-only today: a reinstall loses the money and plan data, and closing that gap is a migration plus a sync path, not a model change.

13 Production edge cases

The register below is the difference between a demo and something you can rely on. Each row is a condition that will occur in normal use, not a theoretical one. The feature chapters cross-reference the rows that apply to them.

13.1 Time and calendars

CONDITION	BEHAVIOUR	STATUS
Sleep spans midnight	Attributed to the <i>wake</i> day via <code>SleepRecord.attributedDate</code>	Designed
Travel across time zones	"Today" follows the device's current zone; a day can be shortened or lengthened	Accepted
DST transition (23h or 25h day)	<code>Calendar.startOfDay</code> handles it; never compute days by adding 86,400 seconds	Designed
Month grid at month boundaries	Leading blanks computed from <code>firstWeekday</code> ; padded to whole rows	Tested
Data arrives for a past day	<code>upsert</code> merges into the existing row; dot grid re-evaluates on next load	Designed
Future-dated day	Rendered <code>.future</code> by position, never evaluated against goals	Tested
Device clock skew vs server	Can cause a stale LWW win	Known limitation

13.2 HealthKit failures

CONDITION	BEHAVIOUR
User denies read access	Undetectable by design. Query anyway; empty result renders an empty state, and Settings links to Health's permission screen
Permission revoked mid-use	Same path: data simply stops arriving; the last-synced timestamp in Settings goes stale
iPhone and Watch both counting steps	De-duplicated by <code>HKStatisticsCollectionQuery</code> . Never sum raw samples
Background delivery doesn't fire	iOS throttles aggressively. Also sync on foreground; never depend solely on background wake
Anchor invalidated (restore from backup)	Anchor reset triggers a full re-read; <code>upsert</code> merge makes this idempotent
Weight recorded in pounds	Read through <code>HKUnit</code> conversion, stored canonically in kg
Watch not worn all day	Fields stay <code>nil</code> . Day is <code>.noData</code> , streak survives
Simulator has no health data	Expected. Seeded data covers development; integration is device-only

13.3 Whoop failures

CONDITION	BEHAVIOUR
Access token expired	Refreshed <i>proactively</i> before each sync (expiry treated as 60s early), via the Edge Function. There is no mid-request refresh-and-retry
Refresh token rotated	Whoop rotates on use, so the <i>new</i> pair is persisted before anything else runs. Reusing an old one invalidates the whole chain
Refresh fails with auth error	Clear Keychain, show "Whoop sign-in expired" in Settings. No silent retry loop. Any other refresh error keeps the tokens
Collection endpoint returns 401	Tokens are kept ; surfaces as "Whoop needs new permissions. Disconnect, then connect again." Usually a token whose scopes predate the current list; clearing it here once made connect → sync an unbreakable loop
Rate limited (429)	Surfaced immediately as "Rate limited by Whoop"; no backoff or retry today. Backoff arrives with the automated sync that could actually loop
Page cap hit (20 pages)	Sync keeps what it has and logs to OSLog; the return value looks like a complete sync, a known open follow-up
Record not yet scored	<code>PENDING_SCORE</code> / <code>UNSCORABLE</code> records are archived but not derived; a later sync's re-derive picks up the scored version
Strap not worn	No recovery record; fields stay <code>nil</code> ; screen shows its empty state and charts show a gap, never a zero
Whoop cycle boundary ≠ calendar day	Cycles are attributed to the day the cycle <i>starts</i> , sleep to the day it <i>ends</i> ; documented so the mismatch isn't read as a bug
Payload shape differs from docs	Raw payloads retained in the local <code>WhoopRawRecord</code> archive; <code>rederive()</code> rebuilds everything without re-fetching. The custom date decoder exists because this already happened: fractional seconds broke stock ISO 8601 parsing on every record
User revokes access in Whoop's app	Next refresh 401s → same reconnect path
Webhook delivered twice planned	Idempotent by record UUID (same upsert keys the sync already uses), so reprocessing is a no-op
Webhook signature invalid planned	Rejected before parsing. Never trust an unverified payload

13.4 Sync failures

CONDITION	BEHAVIOUR
No network	Every screen reads local. Failure surfaces only as a stale timestamp in Settings
Write while offline	Committed locally, <code>syncedAt</code> stays <code>nil</code> , pushed on next successful sync
Partial batch failure	Per-row <code>syncedAt</code> means only unsynced rows retry. No all-or-nothing batches
Same row edited locally and by the nightly job	Field-level LWW: the job's Whoop columns and your water column both survive
Supabase session expires	Renewed silently on launch and on foreground by <code>SessionRefresher</code> (11.3). A refused refresh returns the user to signup; a network or server failure keeps the session and the app keeps working
RLS policy misconfigured	Returns empty, does not throw. Indistinguishable from "no data", so verify policies explicitly
Schema drift SwiftData ↔ Postgres	Decoding failures logged per-row and skipped; one bad row never halts a sync

13.5 UI and data display

CONDITION	BEHAVIOUR
First launch, no data	Seeded 60 days in development; in production every hero shows its designed empty state
Metric missing	Em dash, never <code>0</code> . Rendered by the tile itself, from a <code>String?</code>
Goal changed retroactively	All history re-evaluates on the next <code>load()</code> . Intentional: the grid measures against your current standard
Streak across a no-data gap	Gap is transparent; streak continues
Progress bar with goal of zero	Ratio clamped to 0...1 in the ViewModel; goal ranges in Settings exclude zero
Weight delta with < 7 days history	Delta suppressed entirely rather than computed from a short window
Very large numbers	<code>minimumScaleFactor(0.5)</code> and <code>lineLimit(1)</code> on hero numerals
Dark mode	Every hue has a dark variant; neutral tokens resolve per scheme. No colour defined only inside a scheme branch
Dynamic Type at accessibility sizes	Hero numerals scale down; tiles reflow. Verify at XXL before shipping

13.6 AI failures

V2

CONDITION	BEHAVIOUR
Device lacks Apple Intelligence	<code>SystemLanguageModel.availability</code> checked; coach degrades to unavailable rather than silently uploading health data
Model still downloading	Reported as unavailable with the reason surfaced, not a spinner
Context window exceeded	Tool calling keeps context small by construction; deep analysis routes to the 32K PCC tier
Guardrail refusal	Surfaced plainly. Never retried with a reworded prompt to evade it
Tool returns no rows	Model is told "no data for that range" explicitly, so it says so instead of inventing a trend
User asks about a future date	Tool returns empty; no speculation

14 Operations

14.1 Setup runbook

1 · LOCAL BUILD

```
git clone https://github.com/shivvyas2/LifeOS.git
cd LifeOS
cp Config/Secrets.example.xcconfig Config/Secrets.xcconfig
swift test --package-path LifeOSKit      # no simulator needed
open LIfeOS.xcodeproj
```

In Xcode: Swift Language Version → 6, Minimum Deployments → iOS 26.0, and add `LifeOSKit` via *Add Package Dependencies* → *Add Local*. The app target's folders under `LIfeOS/` are picked up automatically; see 2.3.

2 · SUPABASE

```
supabase init
supabase link --project-ref <ref>
supabase db push                # schema + RLS policies
supabase secrets set WHOOP_CLIENT_SECRET=... \
                             WHOOP_WEBHOOK_SECRET=...
supabase functions deploy whoop-token-exchange
supabase functions deploy whoop-sync
supabase functions deploy whoop-webhook
```

Then put the project URL and **anon** key into `Config/Secrets.xcconfig`.

Phone sign-in needs the SMS provider switched on, because `auth/v1/settings` is what the identity screen asks before it offers the phone tab (4.4). `supabase config push` sends the `[auth.sms]` block from `config.toml` to the linked project. The committed config enables Twilio with *placeholder* SIDs: every number in `[auth.sms.test_otp]` short-circuits before Twilio is called, so the flow is fully exercisable with no account and no message ever sent. Real numbers start working when real credentials replace the zeros — no app change. The CLI refuses to push an enabled provider with a blank SID, which is the only reason the placeholders are shaped like real ones.

3 · WHOOP DEVELOPER APP

1. Register at `developer.whoop.com` (requires an active Whoop membership).
2. Redirect URI: `lifeos://whoop/callback`
3. Request all seven scopes listed in 10.2, including `offline`.
4. Client ID → `Config/Secrets.xcconfig`. Client secret → `supabase secrets set`, **never the xcconfig**.

5. Webhook URL → the deployed `whoop-webhook` function URL.

4 · DEVICE

HealthKit background delivery and the Whoop OAuth round trip cannot be verified in the simulator. Both require a physical device with a signing team configured.

14.2 Secrets

There is no runtime `.env` on iOS. The app is a signed bundle; anything shipped inside it is extractable. The governing rule is therefore: **what must never leak, must live server-side**.

TIER	LOCATION	CONTENTS	IN GIT?
1 · Build config	<code>Config/Secrets.xcconfig</code>	Supabase URL, anon key, Whoop client ID	Gitignored. Template committed
2 · Runtime	Keychain	Whoop access + refresh tokens, Supabase session	Never
3 · Server	Edge Function env	<code>WHOOP_CLIENT_SECRET</code> , <code>WHOOP_WEBHOOK_SECRET</code> , <code>OPENROUTER_API_KEY</code> , <code>SERVICE_ROLE_KEY</code>	Never. Never on device either

Nothing in Tier 1 is truly secret. The Supabase anon key is *designed* to ship in clients, it is safe precisely because RLS gates every row, and the Whoop client ID is public by OAuth design. Tier 1 is gitignored for hygiene, not because a leak would be catastrophic.

SERVICE_ROLE IS THE ONE THAT MATTERS

It bypasses RLS entirely. It belongs only in an Edge Function environment: never in an xcconfig, never in the app target, never in git. Every other secret in this system is recoverable by rotation; this one grants total read and write access to everything.

14.3 Testing strategy

Logic that can be tested without a simulator lives in the package and is developed test-first. ViewModels are tested in the app's own test target against stub stores. SwiftUI view bodies are verified by previews and a build, because asserting on view structure is ceremony that breaks on every layout change without catching real defects.

TARGET	WHAT IS TESTED	HOW
PersistenceTests	Goal evaluation, streak arithmetic, upsert invariants, money rollup, habit ticks	Swift Testing, in-memory ModelContainer
DesignSystemTests	Month grid calendar maths, hue completeness	Swift Testing, fixed UTC calendar
IntegrationsTests	Whoop v2 payload decoding, millisecond timestamps, token refresh, Supabase session restoration	Recorded payload fixtures; stub URLProtocol for session refresh
InsightsTests	On-device routing, escalation policy, brief cache, digest shape	Swift Testing; no network, no model weights
Auth partly written	Session restore, refresh rotation and the refusal rule are covered. Fragment parsing, draft validation and error surfacing are not	Swift Testing; no store or network needed
Feature ViewModels to be written	Empty store, partial data, derivation boundaries	In-memory container; the app target has no test target yet
Views	Both themes, every hue, empty states	SwiftUI previews over snapshot literals
Integrations	Background delivery, OAuth round trip	Manual, physical device

The package declares a macOS platform alongside iOS purely so `swift test --package-path LifeOSKit` runs from the terminal without booting a simulator. Integration modules guard platform-specific imports with `#if canImport(HealthKit)` to preserve this. **219 tests across 31 suites pass today**, all of them in the package. Anything requiring a simulator is, so far, untested by these, though the phone signup path can now be driven by hand in one against the project's test-number map.

THE ONE TEST THAT MATTERS MOST

For every screen: an empty store must produce a snapshot whose readings are all `nil`. It is a four-line test per feature and it is the regression guard against the single worst bug this app can have: a confident zero where there is no data.

15 Cost and the allowance

Life OS ships free, so every user costs money and none of them pay it. That single fact caps the architecture, and it is enforced by a database constraint rather than by good intentions.

15.1 The ceiling

Total running cost must stay under roughly **two dollars per user per calendar month**, covering cloud inference, the API backbone, Supabase, and notifications. There is no revenue per user, so this comes out of pocket and unit economics decide what can be built.

The requirement is that spending *cannot* exceed the allowance, not that it usually does not. Those are different engineering problems, and only the first survives concurrency.

THE ALLOWANCE IS INTERNAL

Users are never shown a balance, a meter, a percentage, or a dollar figure. It exists to bound what the app costs to run, not to be a feature. The only moment it becomes visible is when it runs out, and then it is one sentence.

15.2 Where the money goes

WORKLOAD	TIER FLOOR	WHAT LEAVES THE DEVICE
Daily brief	On-device	Nothing, unless escalated
Coach chat	On-device	Nothing, unless escalated
Calendar assistant	On-device	Nothing, unless escalated
Weekly review	Cloud, reasoning	MetricsDigest : derived numbers only
Meal photo to macros	Cloud, vision	The photo

Three of the five never cost anything unless they escalate for capability. The app's floor is iOS 26, so there is no older-OS cohort that always pays; see 10.1.

THE CALENDAR ASSISTANT WEAKENS THE PRIVACY BOUNDARY, AND SAYS SO

Every other workload escalates a MetricsDigest of derived numbers. The assistant escalates event titles and times, which are content rather than statistics and cannot be aggregated without destroying what makes them useful. This is a genuinely weaker guarantee than the rest of the system offers, it is disclosed once before first use, and it is the reason the assistant floors on-device rather than merely preferring to.

15.3 Why the ceiling holds

The obvious approach does not work:

```
read spend -> if under cap -> call -> add cost
```

Two concurrent requests both read \$1.98 and both proceed. Any scheme that reads and writes in separate statements has this gap, and a ceiling that leaks under concurrency is not a ceiling.

So the proxy **reserves worst-case cost before calling and settles against actual reported usage afterwards**, with the invariant expressed as a check constraint in the schema. A request whose reservation does not fit is refused with `budget_exceeded` before any money is spent. Four properties follow:

PROPERTY	HOW
Concurrency-safe	The constraint is checked by the database, not by application code between two statements
Prices cannot drift from models	Rates are resolved from the provider catalog for whichever model is configured, never hardcoded beside it
Fails closed	No catalog and no cached price means the request is refused, not attempted at a guessed rate. Guessing low is the one path that breaches a ceiling
Bounded loops	Each round of a tool-calling loop reserves separately, so a runaway loop is stopped by the ceiling rather than by trusting its own round cap

AN UNAVAILABLE COACH BEATS AN UNBOUNDED BILL

Refusing on a missing price is deliberate. The refusal surfaces as unavailable rather than as budget exhausted, because they are different problems and deserve different messages.

15.4 What it costs today

On the current demo tier, one flash-class model serves both roles at \$0.10 per million input tokens and \$0.40 per million output.

WORKLOAD	DEMO TIER, PER USER PER MONTH	FRONTIER TIER
Coach, all tasks	~\$0.04	~\$0.87
Calendar assistant	~\$0.02	~\$0.52
Allowance	\$2.00	\$2.00

These are modelled, not measured, and the sensitive input is the escalation rate rather than the per-token price. It is the first thing to check against real data.

WHY NOT THE ACTUAL CHEAPEST MODEL

Providers list models an order of magnitude cheaper than the flash tier. They are smaller than Apple's on-device model, so escalating to one means paying money for a worse answer than the free tier already produced, which inverts the premise of the router. The selection rule is not cheapest available but cheapest model meaningfully more capable than on-device, which in practice means a large context window, since exceeding the on-device context is the expected escalation trigger.

A Product thesis

A.1 The thesis

Every health app you already have is a silo. Whoop knows your recovery but not that you skipped breakfast. Apple Health knows your steps but not that you blew the grocery budget and slept badly because of it. Each one gives advice that is correct in isolation and useless in context.

Life OS is built on a single bet: **the value is in the join, not the collection**. A coach that can see you missed meditation, logged 5 hours of sleep, journalled "low focus," and went 900 calories over can say something no single-purpose app can. That is the entire reason to build one app instead of using five.

Everything in the architecture follows from that bet. There is one table keyed by date that every source writes into and every reader reads from. Cheap manual modules ship early precisely *because* they make the coach smarter, not despite being simple.

A.2 Product PR FAQ

WORKING BACKWARDS · PRODUCT

Life OS is a personal dashboard that pulls together everything your devices already know about you: weight from your scale, recovery from your strap, movement from your watch, and what you ate from a photo. It answers one question every morning: *am I on track?* Open it and a grid of dots shows the month at a glance. Filled means a good day. The app knows what a good day means because you told it.

Who is this for?

One person. This is a personal app, sideloaded on the author's own devices. There are no accounts, no other users, and no App Store distribution. That constraint removes an enormous amount of architecture and is load-bearing throughout this document.

Why not just use Apple Health?

Apple Health is an excellent store and a poor interpreter. It will show you a chart of anything and tell you the meaning of nothing. Life OS reads from it, then adds the layer Health deliberately doesn't: a definition of a good day, a streak that survives days you forgot to log, and eventually a coach that reads across domains.

What is explicitly *not* in this product?

Screen time analysis. It was requested and it cannot be built; see 10.5 for the technical reason. Rather than ship a hollow version, it is cut.

What's the hardest part?

Not the integrations. It's friction: any app that requires daily manual logging dies in about eleven days. Every product decision here is subordinate to keeping the daily cost near zero, which is why food logging is a photo, not a form.

A.3 Roadmap

Each project gets its own spec and plan, and adds or rewrites one chapter here when it lands, not when it is specified. That is why this revision corrects the chapters describing planned work and leaves the feature chapters describing the app as built.

THE V1 TO V7 SLICE SCHEME IS RETIRED

Revision C numbered work V1 to V7. Planning since has been organised as numbered projects with a different ordering, and the two do not map onto each other. The table below is the live one. V1 shipped and is what chapters four to nine describe.

SLICE	CONTENTS	WHY HERE
Shipped	Design system, local data layer, Whoop ingestion and display, Today, Body, Money, Plan, Settings, onboarding	De-risked the hard integrations before adding breadth
1	Health restructure and app-wide restyle: Body becomes Health with Health and Fitness segments; soft light-card aesthetic; custom floating pill nav	UI and IA only, no backend, so it can run alongside other work
0.5	API backbone: a containerised API on GCP Cloud Run, replacing Supabase Edge Functions as the home for server logic. Supabase keeps auth and Postgres	Chosen on cost: free tier at personal scale, and no function-duration limit for long Whoop syncs
2	Nutrition and AI meal logging: MealRecord , meal cards, photo to macros through the coach's vision role	The first feature that needs the cloud tier unconditionally
3	Wake-triggered morning check-in: Whoop webhook to APNs push the moment sleep ends	Needs a real APNs pipeline and a device-token table
4	Calendar layer and AI calendar assistant: EventKit into a SwiftData cache, an agenda card on Today, and the first consumer of tool calling in Insights	Its calendar half depends on nothing; its assistant half needs the coach's Insights skeleton

WHY THE CHEAP MODULES AREN'T AN AFTERTHOUGHT

V4-V7 are each one SwiftData model, one hue token, and one feature folder assembled from existing components. They are cheap *because* V1 builds the design system, the daily spine, and the MVVM template properly. They also carry disproportionate value: every domain added to the join makes the coach materially better at its job.

B Decision log

DECISION	CHOICE	REASON
Audience	Personal only	Removes accounts, App Store review, multi-user architecture
UI architecture	MVVM, one folder per screen	Derivation becomes testable without a simulator; a file's path names its layer
Views and the database	No <code>@Query</code> in a view	Keeps persistence types out of layout code and derivation out of <code>body</code>
Store surface	Protocols returning value types	Stubbable in tests, and nothing above the store can mutate the database
Shared UI	<code>Components/</code> , split <code>Model / ViewModel / View</code>	A shared piece that owns state has an obvious home
Weight source	HealthKit via Renpho's sync	No Renpho API integration needed at all
Whoop	Direct API in V1	Recovery % and Strain exist nowhere else
Whoop client secret	Edge Function only	Would otherwise be extractable from the binary
Food logging	Photo → AI estimate	Lowest friction; friction is what kills these apps
AI coach	On-device Foundation Models by default, escalating to one cloud proxy	Free, private and offline where it can be; capable where it must be, under a ceiling that cannot be breached
Model selection	The app names a role; the proxy maps role to model from environment variables	Upgrading a model is a secrets change, not an App Store release
Provider abstraction	Use Apple's, don't build one	iOS 27 ships <code>LanguageModel1</code> protocols
Data topology	Local-first, Supabase syncs	A health app that waits on the network is one you stop opening
Conflict resolution	Field-level LWW	Row-level would let the nightly job discard your manual entries
Screen time	Cut	Apple's API forbids reading the values into the app
Every metric optional	Yes	A false zero is worse than a blank in a health app
Missing-value rendering	Tiles take <code>String?</code> and render the dash	A rule implemented in nine places is broken in one of them
Dot grid rule	3 of 4, configurable	A vague rule makes the whole grid meaningless
No-data days	Excluded from streaks	Punishing a charging night teaches distrust of the streak
V1 scope	Foundation + health	De-risks the hard integrations before adding breadth
Tab bar	Four tabs: Today, Body, Money, Plan	Four body domains behind one tab; the bar is the scarcest surface in the app

DECISION	CHOICE	REASON
Settings placement	Sheet from a gear on Today	Spends no tab slot on a screen opened once a month
Sign-in	One-time code, no password	No secret at rest and no reset flow to build
Email sign-in	Magic link, temporarily	Free tier cannot put a six-digit token in the template; one constant flips it back
Whoop redirect	https Edge Function bridging to <code>lifeos://</code>	Whoop requires https; the hop holds no secret and PKCE still runs in the app
Whoop sign-in browser	Safari via <code>UIApplication.open</code>	Cloudflare's bot challenge never clears inside <code>ASWebAuthenticationSession</code>
Whoop ingestion order	Archive raw, then derive	A payload never stored cannot be re-derived from; a derivation bug six months out must be fixable without re-fetching against a rate limit
Whoop scopes	Colocated with endpoint paths in <code>WhoopCollection</code>	Listing them separately drifted once and 401'd freshly issued tokens
Whoop collection 401	Keep tokens, tell the user to reconnect	Clearing them here once made connect → sync an unbreakable loop
Whoop sync window	Rolling 14 days, full re-fetch, no cursor	Upserts make re-syncing idempotent, so "which records are new" never needs answering
Sleep attribution	The day the sleep <i>ends</i>	A night belongs to the morning you woke up
Whoop profile scope	Not requested	Name and email are identity, not signal
Plan entries	One model, five kinds	A block renderer can lay out any entry; a new kind is an enum case, not a table
Enum storage	Raw strings with typed accessors	A new case is a code change rather than a migration
Money sign	Positive is money in, opposite to Plaid	Written into the type, because a silent flip makes every rollup lie
Savings rate with no income	<code>nil</code> , not <code>0</code>	Undefined is not zero; the sharpest case of the app's central rule
View model injection	<code>attach(context)</code> , not protocol init	Pragmatic today; it is also why the feature layer has no tests
Reload trigger	<code>ModelContext.didSave</code> plus foregrounding	Event-driven; a save is the only thing that can change what a screen shows

docs/handbook/render-pdf.sh .